

OBJECT ORIENTED PROGRAMMING · CS363

The C++ Objects Handbook

Everything in your seven course slide decks, rewritten from scratch in plain English — with the exact code you need to write, line by line. Start at chapter one knowing nothing.

53 CHAPTERS

BUILT FROM MAT-1 → MAT-7

SIMPLE ENGLISH

FULL SYNTAX

CONTENTS

- I** Ground floor — what C++ is
- II** Two ways to think about a program
- III** Tokens, types and variables
- IV** Operators C++ adds
- V** Functions
- VI** Classes and objects
- VII** Friends and pointers to members
- VIII** Constructors, destructors, operators
- IX** Inheritance
- X** Revision and practice

PART I

Ground floor: what C++ actually is

Before any code, a short story: why programming languages exist, where C++ sits, and how to read your very first program.

CHAPTER 01

How to use this book

This handbook covers exactly the topics in your seven material files — nothing extra, nothing missing. It is written for someone who has never written a line of C++.

Three things you will see on every page

Wherever a new idea appears, it is shown in two boxes next to each other. The left box is the **general form** — the shape you must follow. The right box is a **real example** — that same shape filled in with real names.

● GENERAL FORM

```
return_type function_name(arguments)
{
    // body
}
```

● REAL EXAMPLE

```
int add(int a, int b)
{
    return a + b;
}
```

When a program prints something, the printed result is shown in a strip below it, so you always know what to expect:

OUTPUT

100 + 78 = 178

And three coloured boxes appear throughout:

IN PLAIN WORDS

A short, everyday-English retelling of the idea you just read. If the main text was confusing, read this one.

CAREFUL

A mistake that beginners make here. Read these twice — most marks are lost on exactly these points.

WORTH REMEMBERING

A fact that is asked again and again in exams. If you memorise nothing else, memorise these.

How to study with it

- 1 Read a chapter once, slowly, without trying to memorise anything.
- 2 Type every code example yourself. Do not copy-paste. Typing is how the syntax sticks.
- 3 Break the program on purpose — remove a semicolon, change

```
public
```

```
private
```

 and read the error. Errors teach faster than correct code.

- 4 Come back a day later and read only the coloured boxes. That is your revision.

A NOTE ON WHAT IS COVERED

- Everything here comes from your seven material PDFs on OOP with C++ (Dr. Balu L. Parne, DoCSE, SVNIT Surat; after Balagurusamy and Kanetkar).
- Inheritance, virtual functions, templates, exception handling and file streams are *named* in the materials but not taught in them, so they are not taught here either. They come later in the syllabus.

CHAPTER 02

Where C++ came from

IN THIS CHAPTER

- Why high-level languages were invented
- The four programming paradigms
- Who created which language
- Why anyone still learns C and C++

The ladder from hardware to language

A computer's hardware understands only electricity — on and off. Everything we write has to be turned into that. Over the years, people built a ladder so that humans could write in something readable and the machine could still run it.

LEVEL	WHAT IT LOOKS LIKE	WHO IT SUITS
Hardware	Electrical signals, on/off	The machine only
Machine language	10110000 01100001	Almost nobody
Assembly language	MOV AL, 61h	Hardware engineers
High-level language	x = 97;	Normal programmers
Examples	FORTRAN, C, Pascal, C++	You

IN PLAIN WORDS

Machine language is what the CPU eats. Assembly is machine language with short names instead of numbers. A high-level language lets you write almost-English, and a **compiler** translates it down the ladder for you.

The four paradigms

A **paradigm** is simply a way of thinking about what a program *is*. There are four main ones, and each answers "what is a program?" differently.

PARADIGM	A PROGRAM IS...	LANGUAGES
Imperative	Algorithms + Data	C, Pascal, FORTRAN
Functional	Functions	Lisp, Haskell, ML
Object	Data (objects that carry their own behaviour)	C++, Java, Smalltalk
Logic	Facts + Rules + Queries	Prolog

C++ is mainly in the **object** paradigm, but it still lets you write imperative C-style code. That is why it is called a *multi-paradigm* language, and it is also why this course starts with C-style basics before moving to objects.

Who made what

LANGUAGE	CREATED BY
C	Brian Kernighan and Dennis Ritchie
C++	Bjarne Stroustrup
Java	James Gosling
Python	Guido van Rossum

WORTH REMEMBERING

C++ was created by Bjarne Stroustrup. It began as "C with Classes" — the plan was to keep everything good about C and add the ability to group data and functions together. That is the entire idea of this course.

Why learn C and C++ at all

Your materials give four honest reasons:

- **Core software is written in them.** Operating systems, device drivers, databases, browsers and game engines are C/C++ underneath.
- **Core strengths.** You get speed and direct control over memory. Very few languages let you decide exactly where each byte lives.
- **A huge community.** Decades of libraries, books and answered questions.
- **The STL.** The Standard Template Library gives you ready-made lists, maps, sorting and searching so you do not rebuild them.

CAREFUL

The same power is the danger. C++ will happily let you read memory you do not own or forget to release memory you took. The language will not stop you. That is the trade: full control, full responsibility.

CHAPTER 03

C and C++ standards

A **standard** is an official document that says exactly what the language means. Without it, every compiler would behave differently and your code would not move between machines.

C standards

STANDARD	YEAR	WHAT IT BROUGHT
K&R C	1978	The original C, from Kernighan & Ritchie's book. No official body yet.
C89 / C90	1989/90	The first real standard (ANSI, then ISO). All variables had to be declared at the top of a block.
C95	1995	A small update; added wide-character support.
C99	1999	Big update. You could finally declare a variable anywhere, including inside a <code>for</code> loop.
C11	2011	Multi-threading support, safer library functions.
C18	2018	Bug-fix release. No new features.

C++ standards

STANDARD	YEAR	WHAT IT BROUGHT
C++98	1998	The first ISO standard for C++. The STL becomes official.
C++03	2003	Corrections to C++98.
C++11	2011	The big modernisation: auto , range-for, lambdas, smart pointers.
C++14	2014	Polish on top of C++11.
C++17	2017	Filesystem library, structured bindings.
C++20	2020	Concepts, modules, ranges, coroutines.
C++23	2023	Latest major revision.

IN PLAIN WORDS

You do not need to memorise every year. Remember the shape of it: **C89 was strict about where variables go, C99 relaxed it, C++98 was the first official C++, and C++11 was the big jump forward.** Everything in this course works in every version from C++98 onwards.

CHAPTER 04

Your first C++ program

Here is the smallest complete C++ program that does something. Read it once, then we take it apart line by line.

● HELLO.CPP

```
#include <iostream>
using namespace std;

int main()
{
    cout << "Hello, World";
    return 0;
}
```

OUTPUT

Hello, World

Line by line

LINE	WHAT IT DOES
<code>#include <iostream></code>	Brings in the input/output library. Without it the compiler has never heard of <code>cout</code> . The <code>#</code> means it is handled by the preprocessor , before compiling starts.
<code>using namespace std;</code>	Says "all the standard library names are available without a prefix". Without this line you would write <code>std::cout</code> every time.
<code>int main()</code>	Where every C++ program starts running. <code>int</code> means it hands an integer back to the operating system when it finishes.
<code>{ }</code>	Curly braces mark the start and end of a block. Everything between them is the body of <code>main</code> .
<code>cout << "Hello, World";</code>	Sends the text to the screen. The semicolon ends the statement.
<code>return 0;</code>	Tells the operating system "I finished, and nothing went wrong". Zero means success.

WORTH REMEMBERING

In C++, `main()` returns an `int` to the operating system, so every `main()` should end with `return 0;`. In old C you could write plain `main()` with no return type. Most C++ compilers give a warning or an error if the return is missing.

MAIN() IN C

- `main()` — no return type written
- Return type is `int` by default
- Writing `int` is optional
- Missing return is fine

MAIN() IN C++

- `int main()` — written out
- Ends with `return 0;`

- Compiler expects it
- Missing return warns or errors

Comments

Comments are notes for humans. The compiler ignores them completely.

● BOTH STYLES

```
// This is a single-line comment. It ends at the end of the line.  
  
/* This is a multi-line comment.  
   It keeps going until the closing marker. */
```

IN PLAIN WORDS

The `//` style came from C++ and was so useful that C later copied it. Use it constantly — a comment explaining *why* you did something is worth more than the code itself when you read it a month later.

CHAPTER 05

cout, cin and cascading

The output operator

`cout` is not a keyword or a function. It is a **predefined object** that stands for the standard output stream — normally your screen. A **stream** just means a flow of data.

● GENERAL FORM

```
cout << item;
```

● REAL EXAMPLE

```
cout << "Total: ";  
cout << 250;  
cout << 'A';
```

The `<<` symbol is the **insertion operator**, also called the "put to" operator. It takes whatever is on its right and pushes it into the object on its left.

IN PLAIN WORDS

Read `cout << x;` as "put x into the screen". The arrows point in the direction the data travels — towards `cout`.

The input operator

`cin` is the matching object for the standard input stream — normally your keyboard.

● GENERAL FORM

```
cin >> variable;
```

● REAL EXAMPLE

```
int number1;  
cin >> number1;
```

The `>>` symbol is the **extraction operator**, also called the **"get from" operator**. The program stops and waits for you to type. Whatever you type is stored in the variable on the right.

WORTH REMEMBERING

- `cout` — standard output stream, uses `<<` — insertion / "put to"
- `cin` — standard input stream, uses `>>` — extraction / "get from"
- Both live in the `iostream` header.

Cascading

You do not need a separate statement for each item. Chaining several `<<` or `>>` in one statement is called **cascading**.

● CASCADING BOTH WAYS

```
cout << "sum=" << sum << "\n";

cout << "sum=" << sum << "\n" << "average=" << average << "\n";

cin  >> number1 >> number2;
```

The last line waits for two numbers. You can type them separated by a space or by pressing Enter between them.

A complete program using both

● AVERAGE.CPP

```
#include <iostream>
using namespace std;

int main()
{
    float number1, number2;

    cout << "Enter two numbers: ";
    cin  >> number1 >> number2;

    float sum = number1 + number2;
    float average = sum / 2;

    cout << "sum=" << sum << "\n"
         << "average=" << average << "\n";

    return 0;
}
```

OUTPUT

```
Enter two numbers: 12 18
sum=30
average=15
```

CAREFUL

Use << with cout and >> with cin. Swapping them is the single most common first-week error. A memory hook: the arrows always point *away from* the thing giving data and *towards* the thing receiving it.

CHAPTER 06

namespace std and header conventions

What a namespace is

In C, every library name is **global** — `printf`, `scanf`, `stdin` all sit in one big shared pool of names. If your own function is also called `printf`, there is a clash.

C++ solved this by putting the whole standard library inside a named box called `std`. A box like this is a **namespace**. The full name of the output object is therefore `std::cout`.

WITHOUT USING

- Write `std::` before every standard name
- `std::cout << "Hi" << std::endl;`
- Longer, but never ambiguous

WITH USING NAMESPACE STD;

- Write the short name directly
- `cout << "Hi" << endl;`
- Shorter — used all through this course

● WITHOUT USING

```
#include <iostream>

int main() {
    std::cout << "Hello World in C++"
               << std::endl;
    return 0;
}
```

● WITH USING

```
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World in C++"
         << endl;
    return 0;
}
```

Header file conventions

This trips up a lot of people, so here is the whole rule in one table.

	C HEADER	C++ HEADER
In a C program	Use <code>.h</code> — e.g. <code>#include <stdio.h></code> . Names are global.	Not applicable.
In a C++ program	Prefix <code>c</code> , drop the <code>.h</code> — e.g. <code>#include <cstdio></code> . Names are in <code>std</code> .	No <code>.h</code> at all — e.g. <code>#include <iostream></code> .

● PREFERRED IN C++

```
#include <cmath>    // in C it is <math.h>
...
std::sqrt(5.0);    // needs std::
```

● ALLOWED, NOT PREFERRED

```
#include <math.h>  // not in std namespace
...
sqrt(5.0);        // no std:: needed
```

CAREFUL

Writing a C++ header with a `.h` — like `iostream.h` — is **wrong**. Those forms are deprecated and were removed from the standard long ago. Some old compilers still accept them, which makes the mistake hard to spot. Always write `<iostream>`, never `<iostream.h>`.

CHAPTER 07

C and C++ side by side

Three complete programs written twice — once in C, once in C++. Everything different between them is a thing C++ added.

PROGRAM 1 — SQUARE ROOT OF A NUMBER

● C · SQRT.C

```
#include <stdio.h>
#include <math.h>

int main() {
    double x, sqrt_x;
    printf("Input number:\n");
    scanf("%lf", &x);

    sqrt_x = sqrt(x);

    printf("Sq. Root of %lf is:", x);
    printf(" %lf\n", sqrt_x);
}
```

● C++ · SQRT.CPP

```
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double x;
    cout << "Input number:" << endl;
    cin >> x;

    double sqrt_x = sqrt(x);

    cout << "Sq. Root of " << x;
    cout << " is: " << sqrt_x << endl;
}
```

OUTPUT

C → Input number: 2

C++ → Input number: 2

Square Root of 2.000000 is: 1.414214

Square Root of 2 is: 1.41421

	C	C++
Math header	math.h	cmath
Formatting	You write %lf yourself	Worked out from the variable's type
sqrt function	C standard library	Same C standard library
Default print precision	6 digits	5 digits — different!

PROGRAM 2 — SUM OF N NATURAL NUMBERS

C · SUM_N.C

```
#include <stdio.h>

int main() {
    int n;
    int i;           // must be at top
    int sum = 0;

    printf("Input limit:\n");
    scanf("%d", &n);

    for (i = 0; i ≤ n; ++i)
        sum = sum + i;

    printf("Sum of %d", n);
    printf(" numbers is: %d\n", sum);
}
```

● C++ · SUM_N.CPP

```
#include <iostream>
using namespace std;

int main() {
    int n;
    int sum = 0;

    cout << "Input limit:" << endl;
    cin  >> n;

    for (int i = 0; i ≤ n; ++i)
        sum = sum + i;    // i declared here

    cout << "Sum of " << n;
    cout << " numbers is: " << sum << endl;
}
```

OUTPUT

```
Input limit: 10
Sum of 10 numbers is: 55
```

WORTH REMEMBERING

This is the classic difference. In **C89** the loop counter `i` had to be declared at the beginning of the block. In **C++** (and in C99 onwards) you can declare it right inside the `for` statement. The variable then only exists inside the loop, which is safer.

PROGRAM 3 — CONTROLLING THE PRECISION

C++ prints 5 significant digits by default. To control it, include `<iomanip>` and use `fixed` with `setprecision`.

● PRECISION.CPP

```
#include <iostream>
#include <iomanip>    // needed for setprecision
using namespace std;

int main() {
    double number = 3.141592653589793;

    // 2 decimal places
    cout << fixed << setprecision(2) << number << endl;    // 3.14

    // 4 decimal places
    cout << fixed << setprecision(4) << number << endl;    // 3.1416

    return 0;
}
```

OUTPUT

```
3.14
3.1416
```

The same idea applied to a square root:

● SQRT WITH PRECISION

```
#include <iostream>
#include <cmath>
#include <iomanip>
using namespace std;

int main()
{
    int num;
    double sqrt1;

    cout << "Enter the number" << endl;
    cin >> num;

    sqrt1 = sqrt(num);

    cout << "The square root value of a " << num << " is "
         << fixed << setprecision(2) << sqrt1;

    return 0;
}
```

OUTPUT

```
Enter the number
7
The square root value of a 7 is 2.65
```

PART II

Two ways to think about a program

The whole point of this subject. Old-style programs organise themselves around *actions*. Object oriented programs organise themselves around *things*.

CHAPTER 08

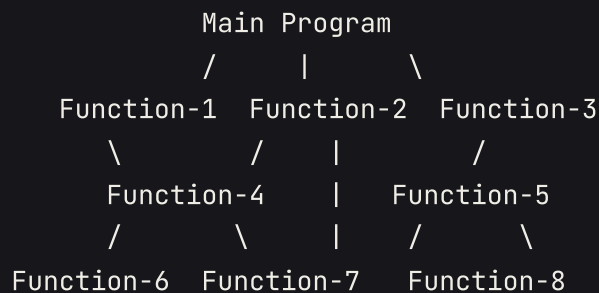
Procedure oriented programming

This is how C works. You write a list of instructions, wrap groups of them in functions, and the functions pass data around between themselves.

The shape of such a program

A main program calls some functions. Those functions call other functions. It forms a tree, and this is called a **top-down** design: you start with the big job and keep breaking it into smaller jobs.

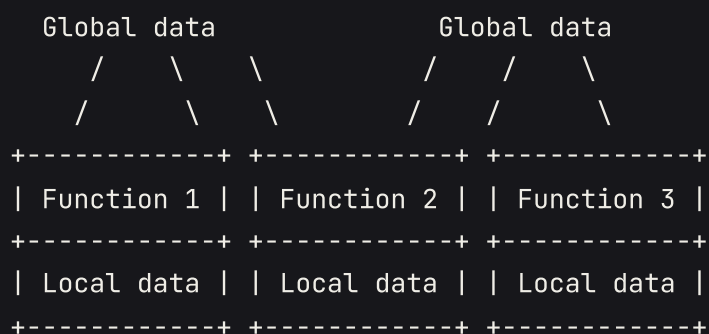
● TYPICAL STRUCTURE



Where the data lives

This is the important part. In a procedural program most data is **global** — it sits outside all the functions, and any function can reach in and change it.

● DATA AND FUNCTIONS IN POP



IN PLAIN WORDS

Imagine a shared whiteboard in an office. Anyone can walk up and rub something out. It works when there are three people. When there are three hundred, and something on the board is wrong, nobody can tell who changed it.

The six characteristics

- **Emphasis is on doing things** — on the algorithm, the sequence of steps.
- **Large programs are divided into smaller programs** called functions.
- **Most functions share global data.**
- **Data moves openly around the system** from function to function.
- **A function transforms data** from one form into another.
- **It uses a top-down approach** in program design.

CAREFUL — THIS IS THE EXAM ANSWER

If asked "what is the main drawback of procedure oriented programming?", the answer is **global data is exposed**. Data moves freely, any function can change it, so a bug in one function can break a completely unrelated part of the program. Object oriented programming exists to fix exactly this.

CHAPTER 09

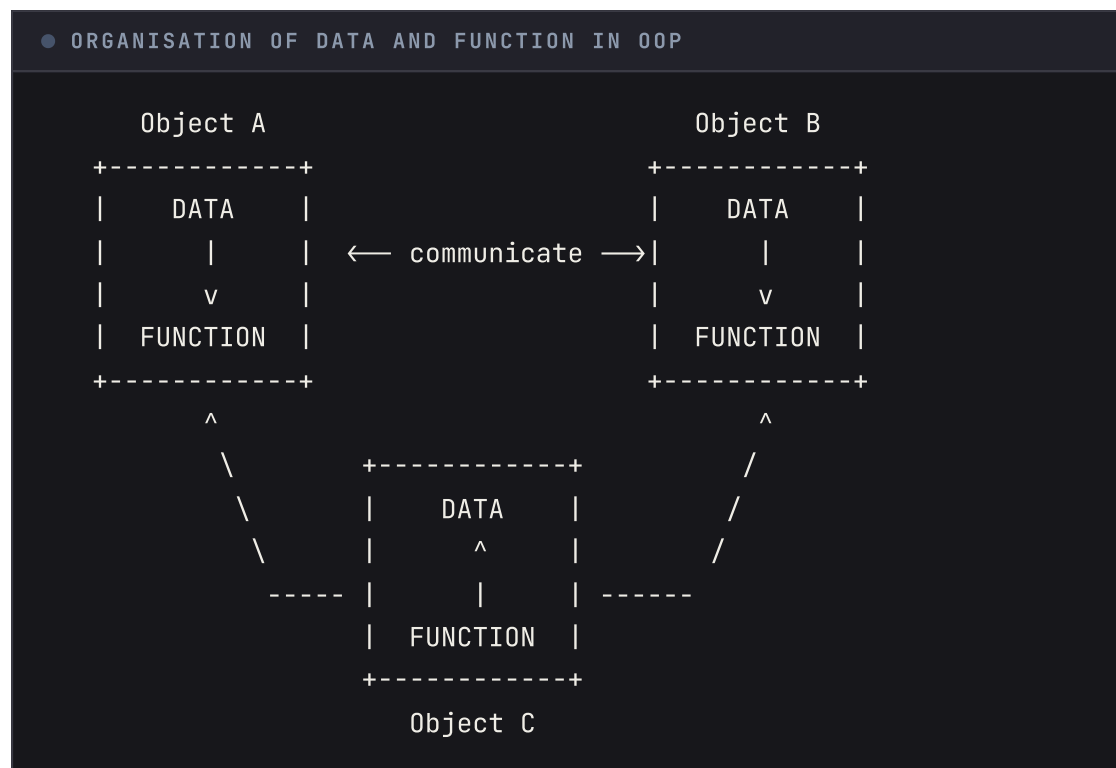
Object oriented programming

OOP turns the picture inside out. Instead of data floating in the open with functions reaching for it, the data is locked inside a box together with the functions that are allowed to touch it.

The four statements that define OOP

- OOP treats **data as the critical element** in program development and **does not allow it to flow freely** around the system.
- OOP **ties data more closely to the functions that operate on it**, and protects it from accidental change by outside functions.
- OOP allows a problem to be broken into a number of entities called **objects**, and then builds data and functions around those objects.
- The data of an object can be reached **only by the functions belonging to that object**. However, functions of one object *can* call functions of other objects.

The picture



Notice the arrow direction inside each box: the function is the only way in or out. Objects talk to each other by **calling each other's functions**, never by reaching into each other's data.

IN PLAIN WORDS

Think of a bank account. Your balance is the data. You are not allowed to open the bank's database and edit your balance. You must call a function — `deposit()` or `withdraw()` — and that function checks the rules first. The data is safe because the only doors into it are guarded.

The seven features of OOP

- **Emphasis is on data** rather than procedure.
- Programs are divided into what are known as **objects**.
- **Data structures are designed such that they characterise the objects** — the data describes the thing.
- Functions that operate on the data of an object are **tied together in the same data structure**.
- **Data is hidden** and cannot be accessed by external functions.
- Objects may **communicate with each other through functions**.
- **New data and functions can be easily added** when needed.

PROCEDURE ORIENTED

- Emphasis on **doing** (the algorithm)
- Divided into **functions**
- Data is **global and shared**
- Data moves openly between functions
- **Top-down** design
- Adding new data means touching many functions

OBJECT ORIENTED

- Emphasis on **data**
- Divided into **objects**
- Data is **hidden inside objects**
- Data reached only through that object's functions
- **Bottom-up** design
- New data and functions added easily

CHAPTER 10

The six core ideas

Your materials list six concepts that are used throughout object oriented programming. Here they are, each with a plain definition and an everyday example.

Classes

Objects

Inheritance

Abstraction

Encapsulation

Polymorphism

1 · Class

A **class** is a user-defined data type that acts as a **blueprint** for creating objects. It wraps data (called **attributes** or **data members**) and the functions that work on that data (called **methods** or **member functions**) into a single unit.

A class lets you model a real-world thing, an idea, or an abstract structure in a tidy, organised way. The complete set of data and code of an object can be made into a user-defined data type using a class. **Classes are the foundation of object oriented programming in C++.**

IN PLAIN WORDS

A class is like an architect's drawing of a house. The drawing says every house of this type has a door, three windows and a roof. But you cannot live in a drawing. You have to build an actual house from it.

2 · Object

An **object** is an **instance of a class** — the actual thing built from the blueprint. It is a concrete representation of the data and behaviour the class described.

To create an object you first define a class with its attributes and methods, then you **instantiate** the class — make one or more objects from it. Each object has its **own separate set of attributes** and can call the class's methods to do work.

IN PLAIN WORDS

The class is the drawing. Objects are the actual houses. House number 12 and house number 14 were built from the same drawing, but house 12 can be painted blue and house 14 green — each object keeps its own values.

3 · Encapsulation

Encapsulation means wrapping the data and the functions that act on it into a single unit, and keeping the data hidden from the outside world. It is the mechanism that makes the whole "data cannot flow freely" promise work.

In C++ you do this with the keyword `private`, which we will meet properly in Chapter 36.

4 · Abstraction

Abstraction means showing only what a user needs and hiding the complicated details behind it. The class gives you a small set of buttons to press and keeps the machinery out of sight.

IN PLAIN WORDS

You drive a car with a steering wheel, an accelerator and a brake. You do not need to know how fuel injection works. The car's controls are an abstraction of a very complicated machine.

Encapsulation is how you hide it. Abstraction is why you hide it. They are two sides of one coin, which is why they are so often confused.

5 · Inheritance

Inheritance is the process by which one class can take on the properties and behaviour of another class. The class that gives is the **base class**; the class that receives is the **derived class**. The derived class gets everything the base had, and can add more of its own.

This is what gives OOP its **reusability** — you write the common part once and reuse it in every class that needs it.

WORTH REMEMBERING

Inheritance is listed as a core concept in your materials, and the protected access specifier exists to support it, but the detailed mechanics of inheritance are **not covered in these seven decks**. They come in a later part of your syllabus. Know the definition; the code comes later.

6 · Polymorphism

Polymorphism is a Greek word meaning "many forms". In programming it means **one name, many behaviours** — the same operation behaves differently depending on what it is applied to.

You will meet two forms of it in this course, and both are taught in full here:

FORM	WHAT IT MEANS	CHAPTER
Function overloading	Several functions share one name; the compiler picks the right one from the arguments.	Chapter 35
Operator overloading	An operator like <code>+</code> is given extra meaning for your own class.	Chapter 48

IN PLAIN WORDS

The word "run" means different things for a person, a program and a business. Same word, different behaviour, and you work out which is meant from the context. Polymorphism is exactly that idea in code.

CHAPTER 11

Benefits and applications

What you gain

- **Data security.** Data is hidden, so outside functions cannot damage it by accident.
- **Reusability.** Through inheritance, code written once is reused in many places.
- **Easy to extend.** New data and new functions can be added to a class without disturbing the rest of the program.
- **Manageable complexity.** A big problem is broken into objects, each small enough to understand on its own.
- **Maps to the real world.** The program's objects match the real things the program is about, so the design is easier to talk about.
- **Easier maintenance.** When something is wrong with a car's behaviour, you look inside the `Car` class and nowhere else.

Where it is used

- Real-time systems
- Simulation and modelling
- Object oriented databases
- AI and expert systems
- Neural networks and parallel programming
- CAD/CAM systems
- Decision support and office automation systems

WORTH REMEMBERING

A very common short-answer question is "list any four benefits of OOP". Pick from: **data hiding and security, reusability through inheritance, easy extension, reduced complexity, closeness to the real world, easier maintenance.** Give one line of explanation for each and you have full marks.

Tokens, types and variables

The raw material of every program. C++ is a superset of C, so most of this will also be true of any C you have seen — but there are important additions, and they are marked.

CHAPTER 12

Tokens and keywords

C++ is a superset of C, which means most C constructs are legal in C++ with their meaning unchanged. There are a few exceptions and several additions, and this part covers them.

What a token is

The **smallest individual units in a program are known as tokens**. When the compiler reads your file, the first thing it does is chop the text into tokens. C++ has five kinds:

Keywords

Identifiers

Constants

Strings

Operators

Your materials sum the whole language up in one line, and it is a good line to remember:

WORTH REMEMBERING

C++ Programs = Tokens + White Spaces + Syntax of the Programming Language + Logic

Keywords

There are **63 keywords** in C++. A keyword (also called a **reserved word**) has a special meaning fixed by the language. You must never use one as the name of your own variable or function. All reserved words are in lower-case letters.

THE ORIGINAL SET			
asm	double	new	switch
auto	else	operator	template
break	enum	private	this
case	extern	protected	throw
catch	float	public	try
char	for	register	typedef
class	friend	return	union
const	goto	short	unsigned
continue	if	signed	virtual
default	inline	sizeof	void
delete	int	static	volatile
do	long	struct	while

ADDED BY ANSI C++			
bool	export	reinterpret_cast	typename
const_cast	false	static_cast	using
dynamic_cast	mutable	true	wchar_t
explicit	namespace	typeid	

IN PLAIN WORDS

Do not try to memorise all 63. Just notice that the ones you already recognise — `if`, `for`, `int` — came from C, and the interesting new ones — `class`, `new`, `delete`, `friend`, `operator`, `public`, `private`, `this`, `virtual` — are exactly the object oriented features this course teaches.

CHAPTER 13

Identifiers and constants

Identifiers

Identifiers are the names of variables, functions, arrays, classes and so on, created by the programmer. Four naming rules are common to both C and C++:

1. Only alphanumeric characters, digits and underscores are permitted.
2. The name cannot start with a digit.
3. Uppercase and lowercase letters are distinct — `Total` and `total` are two different names.
4. A declared keyword cannot be used as a variable name.

WORTH REMEMBERING

One difference is commonly asked: **ANSI C recognises only the first 32 characters of a name. ANSI C++ places no restriction on length** — every character in a name is significant.

● LEGAL NAMES

```
int total;  
int total_marks;  
int totalMarks2;  
int _count;
```

● ILLEGAL NAMES

```
int 2total;      // starts with a digit  
int total marks; // space not allowed  
int total-marks; // hyphen not allowed  
int class;       // keyword
```

Constants

Constants are fixed values that do not change during the execution of a program. C++ supports several kinds of **literal constants** — integers, characters, floating point numbers and strings.

WRITTEN AS	KIND
123	Decimal integer
12.34	Floating point number
037	Octal integer — a leading zero means base 8
0X2	Hexadecimal integer — a leading 0X means base 16
"C++"	String constant — double quotes
'A'	Character constant — single quotes
L'ab'	Wide-character constant — the L prefix

CAREFUL

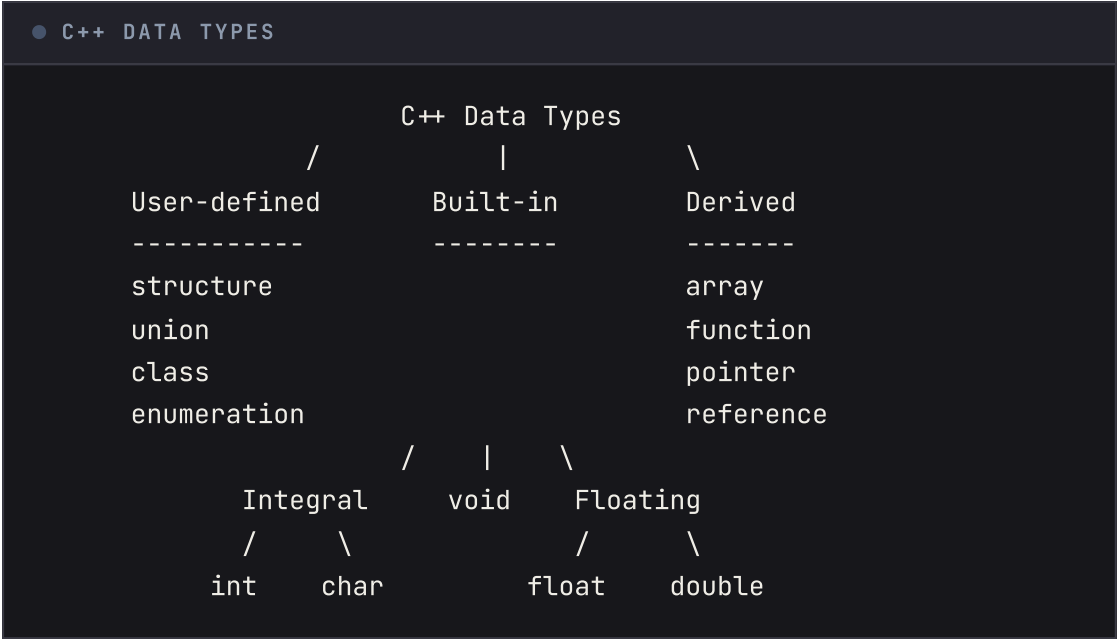
Single quotes and double quotes are not interchangeable. `'A'` is one character. `"A"` is a string of one character plus a hidden terminating `\0`, so it takes two bytes. And watch the leading zero: `037` is octal, which equals 31 in decimal — not 37.

CHAPTER 14

Basic data types

Every variable has a type. The type decides how much memory the variable takes and what can be stored in it.

The family tree



The built-in types and their sizes

DATA TYPE	MEANING	SIZE (BYTES)
int	Integer	2 or 4
float	Floating point	4
double	Double floating point	8
char	Character	1
wchar_t	Wide character	2
bool	Boolean (true / false)	1
void	Empty — no value	0

The full range table

DATA TYPE	SIZE	RANGE
<code>char</code>	1	-128 to 127
<code>unsigned char</code>	1	0 to 255
<code>short</code>	2	-32,768 to 32,767
<code>unsigned short</code>	2	0 to 65,535
<code>int</code>	4	-2,147,483,648 to +2,147,483,647
<code>unsigned int</code>	4	0 to 4,294,967,295
<code>long</code>	4	-2,147,483,648 to +2,147,483,647
<code>unsigned long</code>	4	0 to 4,294,967,295
<code>float</code>	4	-3.4e-38 to +3.4e-38
<code>double</code>	8	1.7e-308 to 1.7e+308
<code>long double</code>	8	1.7e-308 to 1.7e+308
<code>bool</code>	1 bit	true or false
<code>void</code>	—	—
<code>wchar_t</code>	2 or 4	1 wide character

IN PLAIN WORDS

`unsigned` means "no negative values allowed". Because you throw away the negative half, all that room goes to the positive side — which is why `unsigned char` reaches 255 while plain `char` stops at 127.

The void type

The type `void` was introduced in ANSI C. It has two normal uses:

- To specify the **return type** of a function when it does not return any value.
- To indicate an **empty argument list** to a function.

- BOTH USES AT ONCE

```
void funct1(void); // returns nothing, takes nothing
```

There is a third, more interesting use: declaring a **generic pointer**.

- GENERIC POINTER

```
void *gp; // can point at anything

int *ip;
gp = ip; // valid – any pointer can be stored in a void pointer
```

CAREFUL

A generic pointer can be *assigned* a pointer of any basic type, **but it may not be dereferenced**. So `gp = ip;` is valid but `*ip = *gp;` is **invalid**. It would make no sense to dereference a pointer to a void value — the compiler has no idea how many bytes to read or how to interpret them.

CHAPTER 15

User-defined types: struct, union, enum, class

Standalone variables of primitive types are not enough to handle real world problems. You need a way to group related data items together.

An **array** groups together elements of the *same* type. A **structure** groups together elements of *different* types.

Structures

● GENERAL FORM

```
struct name
{
    data_type member1;
    data_type member2;
    ....
};
```

● REAL EXAMPLE

```
struct book
{
    char title[25];
    char author[30];
    int pages;
    float price;
};

struct book book1, book2, book3;
```

You reach the member elements of a structure with the **dot (.)** operator:

● ACCESSING MEMBERS

```
book1.pages = 550;
book2.price = 245.89;
```

IN PLAIN WORDS

A structure is a labelled box. `book1` is one box, and inside it are four labelled compartments. The dot means "inside this box, find this compartment".

Unions

Unions look almost identical to structures — they also group dissimilar elements into a single unit. But their **implementation is completely different**.

The size of a structure is the sum of the sizes of its members. The size of a union is the size of its largest member. That is because in a union, **all members share the same piece of memory**.

● STRUCT — SEPARATE STORAGE

```
struct book
{
    char title[25];
    char author[30];
    int pages;
    float price;
};
struct book book1, book2, book3;
// size = 25 + 30 + 4 + 4 = 63+
```

● UNION — SHARED STORAGE

```
union book
{
    char title[25];
    char author[30];
    int pages;
    float price;
};

// size = 30 (the largest member)
```

	STRUCTURE	UNION
Keyword	<code>struct</code>	<code>union</code>
Size	Greater than or equal to the sum of the sizes of its members	Equal to the size of the largest member
Memory	Each member gets its own unique storage location	The memory allocated is shared by all members
Altering a value	Changing one member does not affect the others	Changing any member alters the other members' values
Accessing	Individual members can be accessed at any time	Only one member can be accessed at a time
Initialisation	Several members can be initialised at once	Only the first member can be initialised

IN PLAIN WORDS

A structure is a chest of drawers — each drawer holds its own thing. A union is a single drawer with several labels on the front. Whatever you put in last is what is inside; the older labels now describe nonsense.

Classes

C++ permits one more user-defined data type: the **class**. It can be used, just like any other basic data type, to declare variables. **The variables of a class are known as objects**, and they are the central focus of object oriented programming.

● GENERAL FORM

```
class ClassName
{
public:
    // Member variables (attributes)
    dataType attributeName1;
    dataType attributeName2;
    // ...

    // Member functions (methods)
    returnType methodName1(parameters);
    returnType methodName2(parameters);
    // ...
};
```

Classes get all of Part VI to themselves. For now, just notice the shape: data and functions inside one pair of braces, ending with a semicolon.

Enumerated data type

An **enumeration** is a user-defined type that provides a way of **attaching names to numbers**, which makes code much easier to read.

The `enum` keyword automatically enumerates a list of words by assigning them the values 0, 1, 2 and so on. This gives you an alternative way of creating symbolic constants.

● DECLARING ENUMERATIONS

```
enum shape    {circle, square, triangle};
enum color    {red, blue, green, yellow};
enum position {off, on};
```

By default the first enumerator gets 0, the second 1, and so on. You can **override the default** by explicitly assigning integer values:

● OVERRIDING THE VALUES

```
enum color {red, blue = 4, green = 8};
// red = 0, blue = 4, green = 8

enum color {red = 5, blue, green};
// red = 5, blue = 6, green = 7 (counting continues from red)
```

WORTH REMEMBERING

When you assign a value to one enumerator, **the ones after it continue counting up from there** unless they are given their own value. This is a favourite exam trick.

CHAPTER 16

Derived types: arrays, functions, pointers, references

Arrays

The use of arrays in C++ is similar to C. **The only exception is the way character arrays are initialised.**

In ANSI C, the compiler lets you declare the array size as the exact length of the string constant — it assumes the programmer means to leave out the null character `\0`. In C++, **the size must be one larger than the number of characters in the string.**

```
● C

char string[3] = "xyz";    // legal in C
```

```
● C++

char string[4] = "xyz";    // room for '\0'
```

WORTH REMEMBERING

This exact pair of lines appears in your material and is a classic one-mark question. **C++ needs room for the terminating null character; C lets you skip it.**

Functions

Functions have undergone major changes in C++. Some of these changes are simple; others require a new way of thinking when organising your programs. Many were driven by the requirements of the object oriented concept of C++, and some were introduced simply to make C++ programs more reliable and readable.

All of those changes are covered in Part V.

Pointers

Pointers are declared and initialised exactly as in C. But in C++ they are **used extensively for memory management and for achieving polymorphism.**

● POINTER BASICS – SAME AS C

```
int x = 10;
int *p;      // p can hold the address of an int
p = &x;      // & gives the address of x
cout << *p;  // * reads the value at that address → 10
```

References

A reference is a derived type that **does not exist in C at all**. It is important enough to get its own chapter — see Chapter 21.

CHAPTER 17

Storage classes

Besides a data type, every variable also has a **storage class**. The storage class specifies the **lifetime** and **visibility** of a variable within the program.

LIFETIME

- The longevity of a variable
- **How long it stays alive** during program execution
- When is it created, when is it destroyed

VISIBILITY

- The scope of a variable
- **Which parts of the program can see it**
- Where can you write its name

auto

- It is the **default** storage class of any type of variable.
- Its visibility is restricted to the function in which it is declared.
- Its lifetime is also limited to the time its container function is executing.
- It is created as soon as its declaration statement is reached, and destroyed as soon as program control leaves its container function block.

extern

- An external variable is declared **outside** of a function but is accessible inside the function block.
- Also called a **global variable**; its visibility spreads all across the program.
- It is accessible by all the functions present in the program.
- The lifetime of an extern variable is the same as the lifetime of the program.

static

- A static variable has the **visibility of a local variable but the lifetime of an external variable**.
- Once declared inside a function block, it does **not get destroyed** after the function finishes — it **retains its value** so it can be used by future function calls.

IN PLAIN WORDS

`static` is the storage class with a memory. An ordinary local variable forgets everything the moment the function ends. A static local variable remembers its value for the next call — useful for a counter that must keep counting.

register

- Similar in behaviour to an automatic variable, but it differs in **where it is stored**.
- Automatic variables are stored in primary memory; **register variables are stored in CPU registers**, which are much faster.
- It is **not an obligation** for a compiler to store a register-type variable only in CPU registers — it is a request, not a command.

All five in one table

STORAGE CLASS	KEYWORD	LIFETIME	VISIBILITY	INITIAL VALUE
Automatic	<code>auto</code>	Function block	Local	Garbage
External	<code>extern</code>	Whole program	Global	Zero
Static	<code>static</code>	Whole program	Local	Zero
Register	<code>register</code>	Function block	Local	Garbage
Mutable	<code>mutable</code>	Class	Local	Garbage

WORTH REMEMBERING

The pattern in the last column is worth noticing: **extern and static start at zero; auto and register start as garbage**. Anything whose lifetime is the whole program gets zeroed for you. Anything short-lived does not.

CHAPTER 18

Symbolic constants

A **symbolic constant** is a name that stands for a fixed value. There are two ways of creating them in C++: using the qualifier `const`, and defining a set of integer constants using `enum`.

Using const

In both C and C++, any value declared as `const` cannot be modified by the program in any way. But there are two important differences between the languages:

	ANSI C	C++
Initialisation	Does not require an initialiser. If none is given, it initialises the <code>const</code> to 0.	Requires a <code>const</code> to be initialised.
Scope	<code>const</code> values are global in nature.	Defaults to internal linkage — local to the file where it is declared.

● MAKING A C++ CONST GLOBAL

```
extern const total = 100; // extern forces external linkage
```

Using enum

Another method of naming integer constants is by enumeration:

● WITH ENUM

```
enum {X, Y, Z};
```

● EQUIVALENT WITH CONST

```
const int X = 0;  
const int Y = 1;  
const int Z = 2;
```

You can also assign values explicitly:

● EXPLICIT VALUES

```
enum {X = 100, Y = 50, Z = 200};
```

CAREFUL

In C++ you **must** initialise a `const` at the point of declaration.

`const int SIZE;` on its own is a compile error in C++, even though a C compiler would quietly set it to 0.

CHAPTER 19

Type compatibility

C++ is very strict with regard to type compatibility compared to C — and there is a specific reason for that strictness.

- C++ defines `int`, `short int` and `long int` as three different types. They must be cast when their values are assigned to one another.
- Similarly, `unsigned char`, `char` and `signed char` are considered as different types, although each of these has a size of one byte.
- In C++, the types of values must be the same for complete compatibility, or else a cast must be applied.

WORTH REMEMBERING — THIS IS THE "WHY"

These restrictions exist for one reason: **to support function overloading**. If `char` and `signed char` were the same type, the compiler could not tell `display(char)` apart from `display(signed char)` and overloading would collapse. Strict types are the price of one name meaning many things. You will see this pay off in Chapter 35.

CHAPTER 20

Declaring and initialising variables

Declare anywhere

In ANSI C, all variables must be declared before they are used in executable statements — usually right at the top of a block. C++ allows the declaration of a variable anywhere in the scope.

● DECLARING WHERE YOU NEED IT

```
int main()
{
    float x;
    float sum = 0;

    for (int i = 1; i < 5; i++) // i declared inside the for
    {
        cin >> x;
        sum = sum + x;
    }

    float average; // declared halfway down
    average = sum / (i - 1);
    cout << average;

    return 0;
}
```

IN PLAIN WORDS

Declare a variable at the place where you first need it, not at the top. The reader then sees the name and its meaning together, and the variable cannot be misused in the fifty lines before it existed.

Dynamic initialisation

In C, a variable must be initialised using a **constant expression**, and the C compiler fixes the initialisation code at the time of compilation.

C++ permits initialisation of variables at run time. This is referred to as dynamic initialisation. A variable can be initialised at run time using an expression at the place of declaration.

● INITIALISING WITH A RUNTIME VALUE

```
....  
int n = strlen(string);           // value known only at run time  
....  
float area = 3.14159 * rad * rad;
```

WORTH REMEMBERING

Dynamic initialisation is used extensively in object oriented programming. It means you can create exactly the type of object needed, using information that is only known at run time. This idea comes back in Chapter 45 as the *dynamic constructor*.

CHAPTER 21

Reference variables

A reference variable provides an **alias** — an alternative name — for a variable that already exists. This is a C++ feature; C does not have it.

How to declare one

A variable is declared as a reference by putting `&` in the declaration. **A reference variable must be initialised at the time of declaration.**

● GENERAL FORM

```
datatype  variable_name;  
datatype &reference_name = variable_name;
```

● REAL EXAMPLE

```
float  total = 100;  
float &sum   = total;
```

Now `sum` and `total` are two names for the **same** piece of memory. Change one and the other changes, because there is only one variable.

● PROOF THAT IT IS ONE VARIABLE

```
#include <iostream>
using namespace std;

int main()
{
    int x = 10;
    int& ref = x;    // ref is a reference to x

    ref = 20;        // value of x is now changed to 20
    cout << "x = " << x << '\n';

    x = 30;          // value of ref is now changed to 30
    cout << "ref = " << ref << '\n';

    return 0;
}
```

OUTPUT

```
x = 20
ref = 30
```

IN PLAIN WORDS

A reference is a nickname. If your name is Rajesh and everyone calls you Raj, there is still only one of you. Writing on Raj's form and writing on Rajesh's form is writing on the same form.

A reference to an array element

● ALIASING ONE SLOT OF AN ARRAY

```
int n[10];
int &x = n[8];    // x is now another name for n[8]
```

The main use: passing arguments to functions

A major application of reference variables is in passing arguments to functions. Normally a function gets a copy of your value and cannot touch your original. With a reference, the function works on your actual variable.

● FUNCTION THAT CHANGES THE CALLER'S VARIABLE

```
void fun(int &x)      // uses reference
{
    x = x + 10;       // x is incremented, so also m
}

int main()
{
    int m = 10;
    fun(m);           // function call - m becomes 20
    ....
}
```

References can be created not only for built-in data types but also for **user-defined data types** such as **structures** and **classes**.

A function can also return a reference

● RETURNING A REFERENCE

```
int &max(int &a, int &b)
{
    if (a > b)
    {
        return a;
    }
    else
    {
        return b;
    }
}
```

Because the return type is `int&`, this function hands back the variable itself, not a copy — which means you can even write `max(x, y) = 0;` and it will assign into whichever of x or y was larger.

CAREFUL

Three rules that catch people out:

- A reference **must** be initialised when declared. `int &r;` alone is an error.
- Once bound, a reference **cannot be re-pointed** at a different variable. Writing `r = y;` copies y's value into whatever r already refers to.
- Never return a reference to a **local** variable of the function — it dies when the function ends, and you are left pointing at nothing.

PART IV

The operators C++ adds

All C operators are valid in C++. On top of them, C++ introduces ten new ones — and several of them only make sense once you have classes, so they come back later.

CHAPTER 22

Operators new in C++

C++ has a rich set of operators. All C operators are valid in C++ also. In addition, C++ introduces the following new operators.

OPERATOR	NAME	WHERE IT IS COVERED
<<	Insertion operator	Chapter 05
>>	Extraction operator	Chapter 05
::	Scope resolution operator	Chapter 23
::*	Pointer to member declarator	Chapter 44
→*	Pointer to member operator	Chapter 44
.*	Pointer to member operator	Chapter 44
delete	Memory release operator	Chapter 24
endl	Line feed operator	Chapter 25
new	Memory allocation operator	Chapter 24
setw	Field width operator	Chapter 25

IN PLAIN WORDS

Six of these ten you can learn right now. The other four — `::*`, `→*`, `.*` and the class-related use of `::` — only have meaning once classes exist, so they are taught in Parts VI and VII where they belong.

CHAPTER 23

The scope resolution operator ::

C++ is a block structured language. The same variable name can be used to mean different things in different blocks — and `::` is how you reach past the nearest one.

The problem

The scope of a variable extends from the point of its declaration till the end of the block containing the declaration. So this is perfectly legal:

```
● TWO SEPARATE BLOCKS

.....
{
    int x = 10;
}

.....
{
    int x = 1;
    .....
}
```

```
● ONE BLOCK INSIDE ANOTHER

.....
{
    int x = 10;
    .....
    {
        int x = 1;    // hides the outer x
        .....
    }
}
```

In the right-hand case, the inner `x` hides the outer one. In C, the global version of a variable cannot be accessed from within the inner block at all. C++ solves this problem by introducing a new operator, `::`, called the **scope resolution operator**.

● GENERAL FORM

`:: variable-name`

This can be used to **uncover a hidden variable**. It gives access to the **global** version of a variable. For example, `::count` means the global version of `count`, and not the local `count` declared in that block.

A complete program

● SCOPE.CPP

```
#include <iostream>
using namespace std;

int m = 10;           // global m

int main()
{
    int m = 20;       // m local to main
    {
        int k = m;    // takes the value of main's m, which is 20
        int m = 30;   // m local to this inner block

        cout << "We are in the inner block \n";
        cout << "k= " << k << "\n";
        cout << "m= " << m << "\n";
        cout << "::m = " << ::m << "\n";
    }

    cout << "\n We are in outer block \n";
    cout << "m = " << m << "\n";
    cout << "::m = " << ::m << "\n";

    return 0;
}
```

OUTPUT

```
We are in the inner block
k= 20
m= 30
::m = 10

We are in outer block
m = 20
::m = 10
```

Reading the output

- `k = 20` — `k` was initialised from `m` *before* the inner `m` was declared, so it picked up main's `m`.
- `m = 30` inside — the innermost declaration wins.
- `::m = 10` everywhere — `::` always jumps straight to the global one, no matter how deep you are.

WORTH REMEMBERING

`::` has a **second, much more important job** in this course: defining a member function outside its class, written as `ClassName::functionName()`. Same operator, different use. That form is taught in Chapter 37, and you will use it constantly.

CHAPTER 24

Memory management: new and delete

We use dynamic allocation techniques when it is not known in advance how much memory space is needed.

Why C++ replaced the C functions

C uses the functions `malloc()`, `calloc()`, `realloc()` and `free()` to allocate and deallocate memory dynamically.

C++ defines two unary operators, `new` and `delete`, to perform the task of allocating and freeing memory in a better and easier way. These operators manipulate memory on the **free store**, so they are also called **free store operators**.

WORTH REMEMBERING — THE KEY SENTENCE

An object can be created by using `new`, and destroyed by using `delete`, as and when required. A data object created inside a block with `new` will remain in existence until it is explicitly destroyed by using `delete`. **The lifetime of such an object is directly under your control and is unrelated to the block structure of the program.**

The new operator

● GENERAL FORM

```
pointer-variable = new data-type;
```

● REAL EXAMPLE

```
int    *p;  
float  *q;  
  
p = new int;  
q = new float;
```

The pointer variable is a pointer of type *data-type*. The `new` operator allocates enough memory to hold a data object of that type, and **returns the address of the object**.

You can combine the declaration and the allocation on one line:

● DECLARE AND ALLOCATE TOGETHER

```
int    *p = new int;  
float  *q = new float;  
  
*p = 25;           // now store values through the pointers  
*q = 7.5;
```

Initialising at the moment of allocation

● GENERAL FORM

```
pointer-variable = new data-type(value);
```

● REAL EXAMPLE

```
int    *p = new int(25);  
float  *q = new float(7.5);
```

Allocating an array

● GENERAL FORM

```
pointer-variable = new data-type[size];
```

● REAL EXAMPLE

```
int *p = new int[10]; // 10 integers
```

The delete operator

Once you no longer need a variable you declared dynamically, you **deallocate** the memory it occupies. The `delete` operator returns the memory to the operating system. This is known as **memory deallocation**.

● GENERAL FORM

```
delete pointer-variable; // single item  
delete [] pointer-variable; // array
```

● REAL EXAMPLE

```
delete p;  
delete q;  
delete [] p;    // for an array
```

The whole cycle in one program

● NEWDELETE.CPP

```
#include <iostream>  
using namespace std;  
  
int main()  
{  
    int* pointVar;  
  
    pointVar = new int;    // ask for memory  
    *pointVar = 45;        // use it  
  
    cout << *pointVar;    // 45  
  
    delete pointVar;      // give it back  
  
    return 0;  
}
```

OUTPUT

45

	C — MALLOC / FREE	C++ — NEW / DELETE
What it is	A library function	An operator, built into the language
Needs a header	Yes — <code>stdlib.h</code>	No
Size	You calculate it: <code>malloc(sizeof(int))</code>	Worked out for you: <code>new int</code>
Return type	<code>void*</code> — usually needs a cast	The correct pointer type already
With objects	Does not call the constructor	Calls the constructor
Freeing	<code>free(p)</code> — no destructor called	<code>delete p</code> — calls the destructor

CAREFUL — THREE CLASSIC ERRORS

- **Forgetting to delete.** The memory is never returned. This is a *memory leak*. The program slowly eats the machine.
- **Using `delete p;` on an array.** For an array you must write `delete [] p;` — the brackets tell C++ to clean up every element, not just the first.
- **Deleting twice.** After `delete p;` the pointer still holds the old address but no longer owns it. Deleting again crashes the program.

CHAPTER 25

Manipulators

Manipulators are operators that are used to format the data display.

The most commonly used ones are `endl` and `setw`.

MANIPULATOR	WHAT IT DOES	HEADER
<code>endl</code>	Feeds the entire line and puts the cursor at the beginning of the next line. You can also use " <code>\n</code> " for the same purpose.	<code>iostream</code>
<code>setw(n)</code>	Sets the minimum field width on output to <code>n</code> characters.	<code>iomanip</code>
<code>setfill(c)</code>	If a value does not fill its field, the character given here is used to fill it. Used together with <code>setw</code> .	<code>iomanip</code>
<code>setprecision(n)</code>	Sets how many digits are shown after the decimal point. Use with <code>fixed</code> .	<code>iomanip</code>

endl

● BASIC OUTPUT WITH ENDL

```
#include <iostream>
using namespace std;

int main()
{
    int    num1 = 70;
    double num2 = 256.783;
    char   ch   = 'A';

    cout << num1 << endl;
    cout << num2 << endl;
    cout << "character: " << ch << endl;

    return 0;
}
```

OUTPUT

```
70
256.783
character: A
```

setw — lining things up in columns

● SALARY.CPP

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    float basic, ta, da, gs;

    basic = 10000; ta = 800; da = 5000;
    gs = basic + ta + da;

    cout << setw(10) << "Basic" << setw(10) << basic << endl
         << setw(10) << "TA" << setw(10) << ta << endl
         << setw(10) << "DA" << setw(10) << da << endl
         << setw(10) << "GS" << setw(10) << gs << endl;

    return 0;
}
```

OUTPUT

```
Basic      10000
   TA         800
   DA        5000
   GS       15800
```

IN PLAIN WORDS

`setw(10)` says "reserve at least 10 characters for the next item, and push it to the right inside that space". That is why the columns line up even though the words and numbers are different lengths. Note that **setw affects only the very next item** — you must repeat it for each one.

setfill — filling the gap

● SETFILL.CPP

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    cout << setw(20) << setfill('*') << "SVNIT"
         << setw(20) << setfill('*') << "Surat" << endl;
}
```

OUTPUT

```
*****SVNIT*****Surat
```

setprecision — controlling decimal places

● PRECISION.CPP

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double number = 3.141592653589793;

    cout << fixed << setprecision(2) << number << endl; // 3.14
    cout << fixed << setprecision(4) << number << endl; // 3.1416

    return 0;
}
```

OUTPUT

```
3.14
3.1416
```

CAREFUL

`setw`, `setfill` and `setprecision` all need `#include <iomanip>`. Only `endl` comes free with `<iostream>`. Forgetting `iomanip` gives a confusing "was not declared in this scope" error.

CHAPTER 26

The type cast operator

C++ allows us to convert data of one type into another. This is known as **type conversion**. There are two types of it.

IMPLICIT CONVERSION

- Done **automatically by the compiler**
- Also known as **automatic conversion**
- You write nothing special

EXPLICIT CONVERSION

- The user **manually changes** the type
- Also known as **type casting**
- You write the cast yourself

Implicit conversion in action

● INT INTO DOUBLE – SAFE

```
int    num_int = 9;
double num_double;

num_double = num_int;

cout << "num_int = "    << num_int    << endl;
cout << "num_double = " << num_double << endl;
```

● DOUBLE INTO INT - LOSES DATA

```
int    num_int;
double num_double = 9.99;

num_int = num_double;    // .99 is thrown away

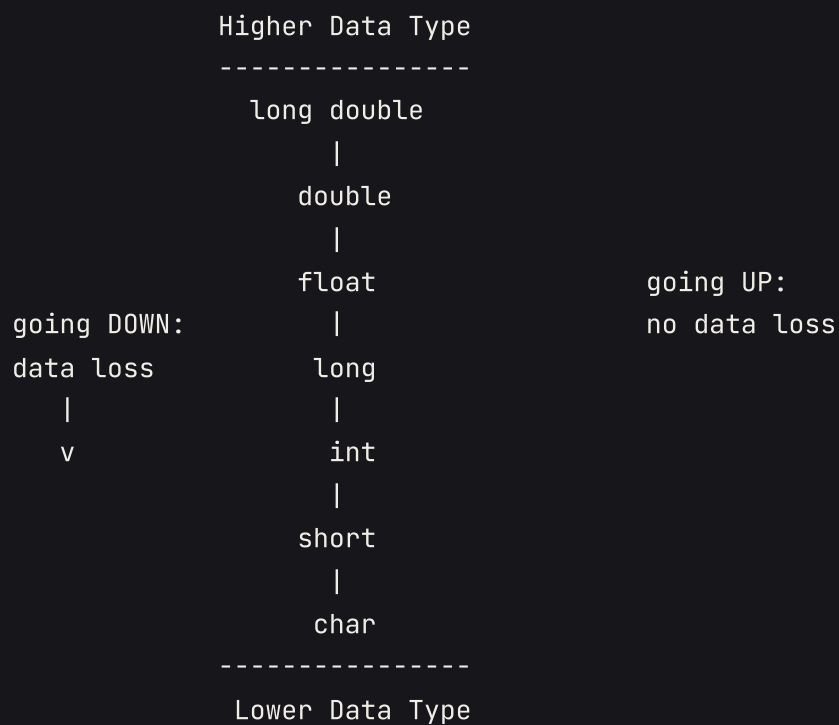
cout << "num_int = "    << num_int    << endl;
cout << "num_double = " << num_double << endl;
```

OUTPUT

Left → num_int = 9 num_double = 9
 Right → num_int = 9 num_double = 9.99 ← the .99 was lost

The direction matters

● THE CONVERSION LADDER



WORTH REMEMBERING

Converting upwards (lower type into higher type) loses nothing.

Converting downwards loses data. A double into an int throws away the fractional part; it does not round it.

The three ways to cast explicitly

There are three major ways in which we can use explicit conversion in C++:

1. **C-style type casting** — also known as cast notation
2. **Function notation** — also known as old C++ style type casting
3. **Type conversion operators**

1 · C-STYLE TYPE CASTING

● GENERAL FORM

```
(data_type) expression;
```

● REAL EXAMPLE

```
int num_int = 26;
double num_double;

num_double = (double) num_int;
```

2 · FUNCTION NOTATION

● GENERAL FORM

```
data_type(expression);
```

● REAL EXAMPLE

```
int num_int = 26;
double num_double;

num_double = double(num_int);
```

BOTH IN ONE PROGRAM

● TYPECAST.CPP

```
#include <iostream>
using namespace std;

int main()
{
    double num_double = 3.56;
    cout << "num_double = " << num_double << endl;

    int num_int1 = (int) num_double;    // C-style
    cout << "num_int1  = " << num_int1  << endl;

    int num_int2 = int(num_double);    // function notation
    cout << "num_int2  = " << num_int2  << endl;

    return 0;
}
```

OUTPUT

```
num_double = 3.56
num_int1    = 3
num_int2    = 3
```

3 · TYPE CONVERSION OPERATORS

The third group is the four named cast keywords C++ added: `static_cast`, `dynamic_cast`, `const_cast` and `reinterpret_cast`. You saw them in the keyword table in Chapter 12.

They are safer than the C style because they say exactly what kind of conversion you intend, so the compiler can refuse a nonsensical one.

IN PLAIN WORDS

For everything in this course, `(int)x` and `int(x)` do exactly the same thing and either is fine. The named casts matter later, when inheritance and pointers make conversions genuinely dangerous.

CHAPTER 27

Expressions and control structures

What an expression is

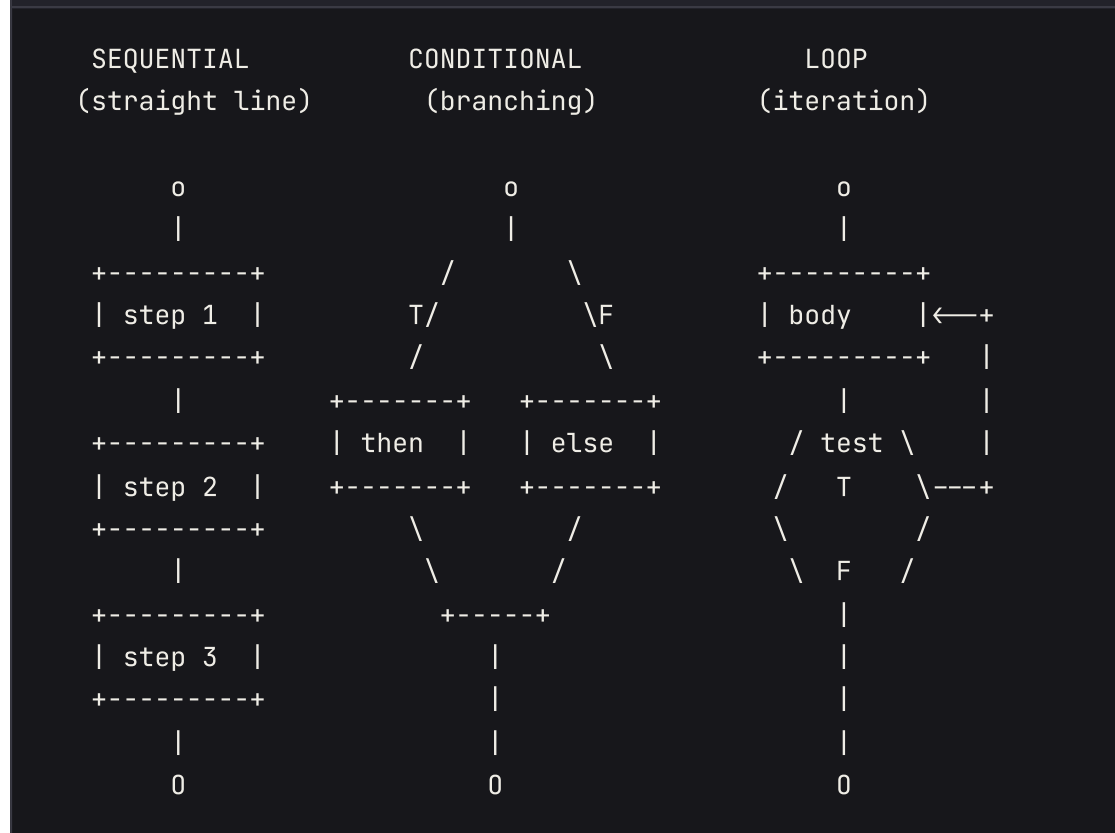
An expression is a combination of operators, constants and variables arranged as per the rules of the language. It may also include function calls which return values. An expression may consist of one or more operands, and zero or more operators, to produce a value.

TYPE OF EXPRESSION	EXAMPLE
Constant expression	<code>20 + 5</code>
Integral expression	<code>m * n - 5</code>
Float expression	<code>x * y / 10</code>
Pointer expression	<code>ptr + 1</code>
Relational expression	<code>x ≤ y</code>
Logical expression	<code>a > b && x == 10</code>
Bitwise expression	<code>x << 3</code>

The three control structures

To fulfil the objective of accurate, error-resistant and maintainable code, you use one or any combination of the following three control structures:

● THE THREE SHAPES OF CONTROL FLOW



- **Sequential structure (straight line)** — one statement after another, top to bottom.
- **Selection structure (branching)** — a test decides which path runs. Built with `if`, `if...else`, `switch` and the `?:` operator.
- **Loop structure (iteration or repetition)** — a block runs again and again while a test holds. Built with `while`, `do...while` and `for`.

The syntax of each

● IF · IF-ELSE

```

if (condition)
{
    // runs when true
}
else
{
    // runs when false
}

```

● SWITCH

```
switch (expression)
{
    case value1:
        // ...
        break;
    case value2:
        // ...
        break;
    default:
        // ...
}
```

● WHILE · DO-WHILE

```
while (condition)
{
    // body - may run zero times
}

do
{
    // body - always runs at least once
}
while (condition);
```

● FOR

```
for (initialise; condition; update)
{
    // body
}

// C++ lets you declare here:
for (int i = 0; i < n; ++i)
{
    // ...
}
```

WORTH REMEMBERING

The one difference worth stating in an exam: `while` tests first, so its body may run zero times. `do...while` tests last, so its body always runs at least once. Note also the semicolon after `while(condition);` at the end of a do-while — it is required there and nowhere else.

PART V

Functions

Functions changed more than anything else when C became C++. Six of the changes in this part — prototyping, references, inline, default arguments and overloading — are pure C++ additions.

CHAPTER 28

Functions, the basics

Dividing a program into functions is one of the major principles of top-down, structured programming. Dividing a complex problem into smaller chunks makes your program easy to understand and reusable.

It is also possible to **reduce the size of the program** by calling and using the same function at different places in the program, instead of writing the same code again and again.

Two types of function

STANDARD LIBRARY FUNCTIONS

- Already **predefined in C++**
- You just include the right header and call them
- Examples: `sqrt()`, `strlen()`, `pow()`

USER-DEFINED FUNCTIONS

- **Created by you**
- You write the definition and then call it
- Everything else in this part

Using a library function

● SQRT FROM CMATH

```
#include <iostream>
#include <cmath>
using namespace std;

int main()
{
    double number, squareRoot;
    number = 25.0;

    squareRoot = sqrt(number);

    cout << "Square root of " << number << " = " << squareRoot;
    return 0;
}
```

OUTPUT

Square root of 25 = 5

Writing your own function

● GENERAL FORM

```
returnType functionName(parameter1,
                        parameter2, ...)
{
    // function body
}
```

● REAL EXAMPLE

```
void greet()
{
    cout << "Hello World";
}
```

Defining and calling

● GREET.CPP

```
#include <iostream>
using namespace std;

// function declaration
void greet()
{
    cout << "Hello World";
}

int main()
{
    // calling a function
    greet();
}
```

The flow is: `main` runs, reaches `greet();`, jumps into `greet`, runs its body, and comes back to the line after the call.

Passing values in

● DISPLAYNUM.CPP

```
#include <iostream>
using namespace std;

void displayNum(int n1, float n2)
{
    cout << "The int number is " << n1 << endl;
    cout << "The double number is " << n2 << endl;
}

int main()
{
    int    num1 = 5;
    double num2 = 5.5;

    // calling the function
    displayNum(num1, num2);

    return 0;
}
```

OUTPUT

The int number is 5
The double number is 5.5

Getting a value back

● ADD.CPP

```
#include <iostream>
using namespace std;

int add(int a, int b)
{
    return (a + b);
}

int main()
{
    int sum;
    sum = add(100, 78);
    cout << "100 + 78 = " << sum << endl;
    return 0;
}
```

OUTPUT

100 + 78 = 178

IN PLAIN WORDS — FOUR WORDS YOU NEED

- **Definition** — the function with its body, the actual code.
- **Call** — the line that says "go and run it now".
- **Formal arguments** — the names in the function header, like a and b.
- **Actual arguments** — the values in the call, like 100 and 78.

CHAPTER 29

main() and function prototyping

The main function

C does not specify any return type for the `main()` function, which is the starting point for the execution of a program.

● MAIN IN C

```
main()
{
    // main program statements
}
```

● MAIN IN C++

```
int main()
{
    // main program statements
    return 0;
}
```

The return type of functions is `int` by default, so the keyword `int` in `main()` is technically optional. But **most C++ compilers will generate an error or warning if there is no `return` statement**. Always write both.

Function prototyping

The **prototype describes the function interface to the compiler** by giving details such as the number and type of arguments and the type of the return value.

- With function prototyping, a **template is always used when declaring and defining a function**.
- When a function is called, the compiler uses the template to ensure that **proper arguments are passed and the return value is treated correctly**.
- **Prototyping was introduced first in C++ by Stroustrup**, and the success of this feature inspired the ANSI C committee to adopt it.

● GENERAL FORM

```
type function-name (argument list);
```

● REAL EXAMPLE

```
float volume (int x, float y, float z);
```

The argument list contains the types and names of arguments that must be passed to the function. **The names are optional** — the compiler only needs the types. All three of these lines mean the same thing to the compiler:

● THREE LEGAL PROTOTYPES

```
float volume (int x, float y, float z);    // names given – clearest
float volume (int, float, float);          // names omitted – legal
float volume (int x, float y, float);      // mixed – also legal
```

CAREFUL

One form that looks right and is **wrong**:

```
float volume(int x, float y, z);
```

Every argument needs its own type. You cannot write `float y, z` and expect `z` to inherit the `float`.

The prototype, the definition and the call

● ALL THREE TOGETHER

```
float volume (int x, float y, float z);    // 1. PROTOTYPE – tells th

int main()
{
    float cube1 = volume(b1, w1, h1);      // 3. CALL
    ....
}

float volume (int a, float b, float c)     // 2. DEFINITION – the rea
{
    float v = a * b * c;
    ....
}
```

Empty argument lists

These two mean exactly the same thing in C++:

● NO ARGUMENTS

```
void display ();  
void display (void);
```

Strict prototype checking

Each of these declarations clearly specifies the function name, the **number, order and type** of arguments each function is going to receive during a call, and the type of the value each function will return.

● THREE PROTOTYPES

```
float square (float);  
char *strConvert (char *, int);  
double nthRoot (float, float);
```

WORTH REMEMBERING

No C++ function can be called unless its prototype is available to the compiler. In C the type checking was a bit relaxed — for example you could pass a double and receive it in a float. C++ will not allow that silently.

Variable number of arguments

It is also possible to pass a variable number of arguments to a C++ function. Such functions require the inclusion of the header file `cstdarg`.

CAREFUL

Heterogeneous types of argument in the list may lead to unexpected output. The call `va_arg` returns a 4-byte sequence from the list if you specified the expected argument to be an integer — **the compiler does not perform type checking** here. Such functions should be used with extreme caution.

CHAPTER 30

The four shapes of a function

Every user-defined function falls into one of four categories, depending on whether it takes arguments and whether it returns a value.

TYPE	TAKES ARGUMENTS?	RETURNS A VALUE?
1	No	No
2	No	Yes
3	Yes	No
4	Yes	Yes

● TYPE 1 – NO ARG, NO RETURN

```
void greet()
{
    cout << "Hello World";
}

greet();
```

● TYPE 2 – NO ARG, RETURNS

```
int getNumber()
{
    int n;
    cin >> n;
    return n;
}

int x = getNumber();
```

● TYPE 3 – TAKES ARG, NO RETURN

```
void displayNum(int n1, float n2)
{
    cout << n1 << " " << n2;
}

displayNum(5, 5.5);
```

● TYPE 4 – TAKES ARG, RETURNS

```
int add(int a, int b)
{
    return (a + b);
}

int sum = add(100, 78);
```

IN PLAIN WORDS

Your materials are honest about this: **there is no hard and fast rule on which method should be chosen.** The particular method is chosen depending upon the situation and how you want to solve a problem. Do not agonise over it — pick the one that makes the code read best.

CHAPTER 31

Call by value, address and reference

This is one of the most important chapters in the course, and one of the most examined. There are three ways to hand a value to a function, and they behave very differently.

Call by value — the traditional way

In a traditional approach, a function call passes arguments **by value**:

- The called function **creates a new set of variables and copies the values** of arguments into them.
- The function **does not have access to the actual variables** in the calling program, and can only work on the copies of values.
- This mechanism is fine if the function does not need to alter the values of the original variables in the calling program.

● CALL BY VALUE — SWAP FAILS

```
#include <iostream>
using namespace std;

void swap(int a, int b)    // a and b are COPIES
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10, y = 20;
    swap(x, y);
    cout << "x = " << x << ", y = " << y;
    return 0;
}
```

OUTPUT

x = 10, y = 20 ← nothing changed!

Call by address — the C solution

Pass the **addresses** of the variables instead of their values, using pointers.

● CALL BY ADDRESS — SWAP WORKS

```
#include <iostream>
using namespace std;

void swap(int *a, int *b)    // a and b hold ADDRESSES
{
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main()
{
    int x = 10, y = 20;
    swap(&x, &y);           // must write & at the call
    cout << "x = " << x << ", y = " << y;
    return 0;
}
```

OUTPUT

x = 20, y = 10

Call by reference — the C++ solution

The provision of the reference variables in C++ permits us to pass parameters to the functions by reference. When we pass arguments by reference, the formal arguments in the called function become aliases to the actual arguments in the calling function.

● CALL BY REFERENCE – SWAP WORKS, CLEANLY

```
#include <iostream>
using namespace std;

void swap(int &a, int &b)    // a and b are ALIASES for x and y
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10, y = 20;
    swap(x, y);             // looks exactly like call by value
    cout << "x = " << x << ", y = " << y;
    return 0;
}
```

OUTPUT

x = 20, y = 10

All three compared

	CALL BY VALUE	CALL BY ADDRESS	CALL BY REFERENCE
What is passed	A copy of the value	The address	An alias for the variable
Header looks like	<code>f(int a)</code>	<code>f(int *a)</code>	<code>f(int &a)</code>
Call looks like	<code>f(x);</code>	<code>f(&x);</code>	<code>f(x);</code>
Inside, you write	<code>a</code>	<code>*a</code>	<code>a</code>
Changes the original?	No	Yes	Yes
Available in	C and C++	C and C++	C++ only

WORTH REMEMBERING

The favourite exam question is *"write a C++ program to swap the values of two numbers using call by value, call by address, and call by reference variable"* — exactly as it appears in your material. All three programs are above. Note the tell-tale difference: **call by reference and call by value look identical at the call site**. Only the function header tells you which is happening.

CAREFUL

Because the call looks the same, a reader of `swap(x, y);` cannot tell whether `x` is about to be modified. This is the one real downside of call by reference, and it is why `const` references exist — see Chapter 43.

CHAPTER 32

Inline functions

Functions save memory space, which becomes appreciable when a function is likely to be called many times. But every call costs time. Inline functions are the fix.

What a function call actually costs

Every time a function is called it executes instructions such as:

- Jumping to the function
- Saving registers, pushing arguments into the stack
- Returning to the calling function

Spelled out in more detail, this is:

STEP	WHAT HAPPENS
Function call overhead	The program needs to push arguments onto the stack, transfer control to the function, execute the function code, and then return control to the caller.
Stack operations	Pushing and popping function parameters and return addresses, which can be relatively costly in terms of execution time.
Context switching	The CPU needs to save the current state (context) and restore it after the function call.

When a function is small, a substantial percentage of execution time may be spent in such overheads.

The old solution: macros — and why they are bad

● A MACRO

```
#define Square(x) (x*x)

int y = Square(3+1);    // expands to (3+1*3+1) = 7, NOT 16!
```

Use of **macros** is one solution to the overhead problem. However, **macros are not really functions and therefore the usual error checking does not occur during compilation.**

The C++ solution: inline functions

To eliminate the cost of calls to small functions, C++ proposes a new feature called the **inline function**.

An inline function is a function that is expanded in line when it is invoked — that is, the compiler replaces the function call with the corresponding function code.

● GENERAL FORM

```
inline function-header
{
    function body
}
```

● REAL EXAMPLE

```
inline double cube(double a)
{
    return (a * a * a);
}

c = cube(3.0);           // becomes 3.0*3.0*3.0
d = cube(2.5 + 1.5);     // correct, unlike a macro
```

Before and after

● NORMAL FUNCTION — 4 STEPS PER CALL

```
int add(int a, int b)
{
    return a + b;
}

int main()
{
    int x = 5, y = 10;
    int result = add(x, y);
    cout << "Result: " << result;
    return 0;
}
```

● INLINE – NO CALL AT ALL

```
inline int add(int a, int b)
{
    return a + b;
}

int main()
{
    int x = 5, y = 10;
    int result = add(x, y);    // becomes x + y
    cout << "Result: " << result;
    return 0;
}
```

On the left, calling `add(x, y)` involves: 1. pushing `x` and `y` onto the stack, 2. transferring control to `add`, 3. executing `return a + b`, 4. returning to `main`. On the right, the compiler replaces the call with `x + y` directly inside `main`. There is no overhead.

A complete program

● `INLINE.CPP`

```
#include <iostream>
using namespace std;

inline float mul(float x, float y)
{
    return (x * y);
}

inline double div(double p, double q)
{
    return (p / q);
}

int main()
{
    float a = 34.56;
    float b = 9.82;

    cout << mul(a, b) << endl;
    cout << div(a, b) << endl;

    return 0;
}
```

The rules

- To make a function inline, **prefix the keyword `inline` to the function definition.**
- **All inline functions must be defined before they are called.**
- The speed benefits of inline functions **diminish as the function grows in size.**
- At some point the overhead of the function call becomes small compared to the execution of the function, and the benefits of inlining may be lost.
- Hence, **usually functions are made inline when they are small enough to be defined in one or two lines.**

WHEN TO USE INLINE

- Small, frequently called functions
- Where call overhead is significant compared to the function's execution time
- Simple getters and setters
- Small mathematical operations

THE DRAWBACK: CODE BLOAT

- If the inline function is large or called many times, each call gets its own copy of the code
- This increases binary size
- Which can *hurt* performance through increased cache usage and memory footprint

WORTH REMEMBERING

`inline` is a **request, not a command**. The compiler may ignore it — for example if the function has a loop, a switch, or is recursive. Ask for it; do not assume you got it.

CHAPTER 33

Default arguments

C++ lets you give a function argument a default value, so that callers who do not care about it can simply leave it out.

● DEFAULTARGS.CPP

```
#include <iostream>
using namespace std;

int sum(int n1, int n2, int n3 = 0, int n4 = 0);    // defaults go HERE

int main()
{
    int s1, s2, s3;

    s1 = sum(10, 20);                // n3 and n4 become 0
    s2 = sum(10, 20, 30);            // n4 becomes 0
    s3 = sum(10, 20, 30, 40);        // nothing defaulted

    cout << s1 << endl << s2 << endl << s3 << endl;
    return 0;
}

int sum(int n1, int n2, int n3, int n4)           // NOT here
{
    return (n1 + n2 + n3 + n4);
}
```

OUTPUT

```
30
60
100
```

The three rules

1. The default values are mentioned only in the function prototype, and not in the function definition. The compiler uses the prototype information to build a call, not the function definition.

2. **Default values can be assigned only for trailing arguments** — the ones at the end — and not to leading arguments or arguments in the middle.
3. The default value for an argument can be a **global constant, a global variable, or even a function call**.

● ILLEGAL — GAP IN THE MIDDLE

```
int sum(int n1, int n2, int n3 = 0, int n4);
```

● LEGAL — A FUNCTION CALL AS DEFAULT

```
int myfunc (int flag = display());
```

IN PLAIN WORDS — WHY RULE 2 EXISTS

Suppose `sum(int a, int b = 5, int c)` were legal, and someone wrote `sum(1, 2)`. Is 2 the value of b or of c? The compiler cannot tell. Arguments are matched left to right, so the ones you may omit must all be at the right-hand end.

When default arguments are useful

Your materials give two specific cases:

- **When you do not want the trouble of writing arguments that almost always have the same value.** For example, bank interest will remain the same for all customers for a particular period of deposit.
- **When you want to extend an existing function.** After having written a program, you may decide to increase the capability of a function by adding another argument. Using default arguments means that the existing function calls can continue to use the old number of arguments, while new function calls can use more.

WORTH REMEMBERING

The second reason is the important one in real software: **default arguments let you add a parameter without breaking any code that already calls the function.** That is a genuinely valuable property and a good exam answer.

CHAPTER 34

Recursion

A **recursive** function is one that calls itself. Your materials give two classic examples: calculating the factorial of a number, and solving the Tower of Hanoi problem.

Factorial

● FACTORIAL.CPP

```
#include <iostream>
using namespace std;

long fact (int n)
{
    if (n == 0)
    {
        return 1;           // base case – stops the recursion
    }
    else
    {
        return (n * fact(n-1)); // recursive case – calls itself
    }
}

int main()
{
    int num;
    cout << "Enter a positive integer: ";
    cin  >> num;
    cout << "Factorial of " << num << " is " << fact(num);
    return 0;
}
```

OUTPUT

```
Enter a positive integer: 5
Factorial of 5 is 120
```

How it unwinds

● TRACING FACT(5)

```
fact(5) = 5 * fact(4)
           fact(4) = 4 * fact(3)
                   fact(3) = 3 * fact(2)
                           fact(2) = 2 * fact(1)
                                   fact(1) = 1

                                   fact(1) = 1
                           fact(2) = 2 * 1 = 2
                   fact(3) = 3 * 2 = 6
           fact(4) = 4 * 6 = 24
fact(5) = 5 * 24 = 120
```

WORTH REMEMBERING — EVERY RECURSIVE FUNCTION NEEDS TWO THINGS

- A **base case** — a condition where it stops calling itself and simply returns. Here it is `if (n == 0) return 1;`
- A **recursive case** that moves *towards* the base case. Here `fact(n-1)` makes *n* smaller each time.

Leave out the base case and the function calls itself forever until the stack runs out. That crash is called a **stack overflow**.

Tower of Hanoi

The second classic. With 3 disks on tower 1, moving them all to tower 2 takes these steps:

● SOLUTION FOR 3 DISKS

```
There are 3 disks in tower 1
Shift top disk from tower 1 to tower 2
Shift top disk from tower 1 to tower 3
Shift top disk from tower 2 to tower 3
Shift top disk from tower 1 to tower 2
Shift top disk from tower 3 to tower 1
Shift top disk from tower 3 to tower 2
Shift top disk from tower 1 to tower 2

3 disks in tower 1 are shifted to tower 2.
```

Seven moves for three disks. The pattern is $2^n - 1$ moves for n disks — which is why the puzzle becomes impossible very quickly as n grows.

CHAPTER 35

Function overloading

This is your first taste of real polymorphism, and it is a pure C++ feature.

The problem

In C, every function in a program has a unique name. However, in some situations it seems unnecessary to have different function names if the task performed by them is the same.

● THREE FUNCTIONS DOING THE SAME JOB

```
int    absolute(int i);  
long   absolute(long l);  
double absolute(double d);
```

C++ overcomes this situation by allowing the programmer to create three different functions with the same name. This is called function overloading.

How the compiler chooses

How does the C++ compiler know which of the `absolute` functions should be called when a call is made? It decides that from the type of argument being passed during the function call.

● TYPES THE COMPILER WAS NOT GIVEN

```
char ch = absolute('A');    // no char version exists  
float f = absolute(3.14f);  // no float version exists
```

As we have not defined the `absolute` function to handle the character and floating point value, the C++ compiler **calls the integer version for the character, and the double version for the floating value**. It promotes the argument to the nearest type it does have.

IN PLAIN WORDS

The compiler plays matchmaker. It looks at what you passed and finds the closest available function. An exact match wins. If there is no exact match, it will promote a smaller type to a bigger one — `char` becomes `int`, `float` becomes `double`.

Rule 1 — the return type is not enough

The overloaded functions must at least differ in the type, number or order of parameters they accept. Difference only in return type is not enough to consider them as overloaded functions.

● THIS WILL NOT COMPILE

```
#include <iostream>
using namespace std;

int    stringToNumber (char *);    // same parameters,
long int stringToNumber (char *);  // only return type differs –

int main()
{
    int    n1;
    long int n2;
    char *ptr1 = "155";
    char *ptr2 = "400000";

    n1 = stringToNumber (ptr1);
    n2 = stringToNumber (ptr2);
    cout << n1 << endl << n2;
    return 0;
}
```

CAREFUL — WHY THIS FAILS

Think about the call `stringToNumber(ptr1);` written on its own line, with the result thrown away. The compiler sees only the argument, which is identical for both versions. It has no way to choose. So **C++ forbids overloading on return type alone.**

Rule 2 — typedef does not create a new type

Can we fool the C++ compiler into believing that two data types are different by renaming one of them using `typedef`?

No. A `typedef` merely gives another name for an existing type and does not constitute a different type.

● ALSO WILL NOT COMPILE

```
typedef int INT;

void display(int);
void display(INT);    // INT is just another name for int – ERROR
```

Here `INT` is just another name for an `int`. Hence the compiler **cannot differentiate between the two versions of `display()`**.

Rule 3 — `const` does create a difference

This one *does* work. A pointer to a string and a pointer to a constant string are genuinely different types.

● OVERLOADING ON CONST – LEGAL

```
#include <iostream>
using namespace std;

void display (char *);
void display (const char *);

int main()
{
    char      *ch1 = "Hello";
    const char *ch2 = "Bye";

    display (ch1);    // calls the first version
    display (ch2);    // calls the second version
    return 0;
}

void display (char *p)
{
    cout << p << endl;
}

void display (const char *p)
{
    cout << p << endl;
}
```

The first version of `display()` receives a pointer to a string, whereas the second receives a pointer to a **constant** string.

CAN YOU OVERLOAD ON...	YES / NO	WHY
Different number of parameters	Yes	The call itself shows how many you passed
Different types of parameters	Yes	The compiler reads the types at the call
Different order of parameters	Yes	e.g. <code>(int, float)</code> vs <code>(float, int)</code>
const on a pointer parameter	Yes	<code>char*</code> and <code>const char*</code> are different types
Return type only	No	The call does not reveal the return type
A typedef name	No	<code>typedef</code> renames, it does not create a type

Practice problems from your material

TRY THESE TWO

- Write a program to compute the area of a square, a rectangle and a circle by overloading the `area()` function.
- Write a function `power()` to raise a number m to a power n . The function takes a double value for m and an `int` value for n , and returns the result correctly. Use a default value of 2 for n to make the function calculate the square when this argument is omitted. Write a main function that gets the values of m and n from the user to test the function. Use the concept of function overloading.

Here is the first one worked out, so you can check your approach:

● AREA.CPP — OVERLOADED THREE WAYS

```
#include <iostream>
using namespace std;

float area(float side)           // square
{
    return side * side;
}

float area(float length, float width) // rectangle
{
    return length * width;
}

double area(double radius, int flag) // circle
{
    return 3.14159 * radius * radius;
}

int main()
{
    cout << "Square      : " << area(5.0f)      << endl;
    cout << "Rectangle : " << area(5.0f, 3.0f) << endl;
    cout << "Circle     : " << area(7.0, 0)      << endl;
    return 0;
}
```

OUTPUT

```
Square      : 25
Rectangle   : 15
Circle      : 153.938
```

CAREFUL

Notice the awkward extra `int flag` on the circle version. Without it, `area(float)` and `area(double)` would both be candidates for a single number and the call would be ambiguous. Real overload sets need parameter lists that are genuinely distinguishable — that is the whole skill.

Friend and virtual functions

Your material closes this topic by naming two more function types C++ introduces: **friend** and **virtual** functions. These are basically introduced to handle some specific tasks related to class objects.

Friend functions get a full chapter — Chapter 42. Virtual functions belong to inheritance and come later in your syllabus.

PART VI

Classes and objects

Everything so far has been preparation. This is the actual subject: bundling data with the functions that guard it, and then making objects from that bundle.

CHAPTER 36

Declaring a class

A class is a user-defined data type that serves as a blueprint for creating objects. It encapsulates data (attributes) and methods (functions) that operate on that data within a single unit.

The general form

● GENERAL FORM

```
class ClassName
{
public:
    // Member variables (attributes)
    dataType attributeName1;
    dataType attributeName2;
    // ...

    // Member functions (methods)
    returnType methodName1(parameters);
    returnType methodName2(parameters);
    // ...
};
```

PART	WHAT IT MEANS
<code>class</code>	The keyword used to declare a class.
<code>ClassName</code>	The name of the class. Follows the same rules as variable naming.
<code>public:</code>	An access specifier. <code>public</code> makes the members accessible from outside the class.
<code>private:</code> , <code>protected:</code>	Other access specifiers. <code>private</code> makes members accessible only within the class itself; <code>protected</code> allows access from within the class and its derived classes.
<code>dataType</code>	The data type of the member variable, or the return type of the member function.
<code>attributeName1</code>	Names of member variables (attributes).
<code>returnType</code>	The data type returned by the member function.
<code>methodName1</code>	Names of member functions (methods).
<code>parameters</code>	The input parameters the member function may take.

CAREFUL — THE MOST COMMON SYNTAX ERROR IN THE WHOLE COURSE

A class declaration ends with a semicolon after the closing brace: `};` — not just `}`. Forgetting it produces a wall of confusing errors that point at the *next* line, not at the class.

The complete example from your material

● THE ITEM CLASS

```
class item
{
    int    number;    // variables declaration
    float cost;       // private by default

public:
    void getdata(int a, float b); // functions declaration
    void putdata(void);           // using prototype
};
```

public vs private — the whole point of OOP

Notice that `number` and `cost` appear *before* any access specifier. In a class, members are **private** by default. That means only the class's own functions can touch them.

PRIVATE

- The **default** in a class
- Accessible **only within the class itself**
- Objects **cannot** reach these members
- This is where your data belongs

PUBLIC

- Must be written explicitly
- Accessible **from outside the class**
- Objects **can** call these
- This is where your functions belong

Creating objects

Once the class exists, you create objects from it. There are two ways.

● WAY 1 — AFTER THE CLASS

```
class item
{
    // ...
};

item x;           // one object
item x, y, z;     // three objects
```

● WAY 2 — IN THE DECLARATION

```
class item
{
    int    number;
    float cost;
public:
    void getdata(int a, float b);
    void putdata(void);
} x, y, z;        // objects created here
```

What an object may and may not do

● LEGAL

```
x.getdata(100, 75.5);  
x.putdata();
```

● ILLEGAL

```
x.number = 100;    // number is private  
x.cost   = 75.5;   // cost is private
```

WORTH REMEMBERING — STATE IT IN EXACTLY THESE WORDS

Objects cannot access the private members of a class directly. They must go through a public member function. That is encapsulation working. In the example, `x` cannot set `number` itself, but it can ask `getdata()` to do it — and `getdata()` could check the value first.

A full working class

● RECTANGLE.CPP

```
#include <iostream>
using namespace std;

class Rectangle
{
public:
    int width;
    int height;

    int calculateArea()
    {
        return width * height;
    }

    void setDimensions(int w, int h)
    {
        width = w;
        height = h;
    }
};

int main() {
    Rectangle rect1, rect2;

    rect1.setDimensions(5, 10);
    rect2.setDimensions(3, 7);

    int area1 = rect1.calculateArea();    // 50
    int area2 = rect2.calculateArea();    // 21

    cout << "Area of Rectangle 1 is : \n" << area1 << endl;
    cout << "Area of Rectangle 2 is : \n" << area2 << endl;

    return 0;
}
```

OUTPUT

```
Area of Rectangle 1 is :
50
Area of Rectangle 2 is :
21
```

Notice the key fact: `rect1` and `rect2` were made from the same class, but each **has its own separate width and height**. Setting `rect1` to 5×10 did nothing to `rect2`.

Objects with strings

● CAR.CPP

```
#include <iostream>

// Define a class named 'Car'
class Car {
public:
    // Attributes (data members)
    std::string brand;
    std::string model;
    int year;

    // Method to display information about the car
    void displayInfo() {
        std::cout << "Brand: " << brand << ", Model: " << model
            << ", Year: " << year << std::endl;
    }
};

int main() {
    // Create objects of the 'Car' class
    Car car1;
    Car car2;

    // Set attributes for each object
    car1.brand = "Toyota";
    car1.model = "Camry";
    car1.year = 2020;

    car2.brand = "Honda";
    car2.model = "Accord";
    car2.year = 2019;

    // Call the method to display information about each car
    car1.displayInfo();
    car2.displayInfo();

    return 0;
}
```

OUTPUT

Brand: Toyota, Model: Camry, Year: 2020

Brand: Honda, Model: Accord, Year: 2019

CHAPTER 37

Member functions

There are two ways to write the body of a member function: **outside the class definition** or **inside the class definition**. Both produce identical behaviour.

Outside the class — using ::

This is the form you will use most, because it keeps the class declaration short and readable.

● GENERAL FORM

```
return-type class-name :: function-name
                        (argument declaration)
{
    Function body
}
```

● REAL EXAMPLE

```
void item :: getdata(int a, float b)
{
    number = a;
    cost   = b;
}
```

The full example from your material:

● BOTH MEMBER FUNCTIONS DEFINED OUTSIDE

```
void item :: getdata(int a, float b)
{
    number = a;
    cost   = b;
}

void item :: putdata(void)
{
    cout << "Number :" << number << endl;
    cout << "Cost  :"  << cost   << endl;
}
```

The **membership label** `item ::` tells the compiler that this function belongs to the class `item`. That is what gives the function permission to touch the private members `number` and `cost`.

IN PLAIN WORDS

Think of `item :: getdata` as a full postal address and `getdata` as just a house number. Many classes might have a `getdata`; the `item ::` part says which one this is.

THREE CHARACTERISTICS OF A MEMBER FUNCTION DEFINED OUTSIDE

- Several different classes can use the same function name. The membership label resolves the scope.
- Member functions can access the **private** data of the class. A non-member function cannot.
- A member function can call another member function directly, without using the dot operator.

Inside the class — automatically inline

When you write the body inside the class definition, the function is treated as an **inline function** automatically. You do not write the keyword.

● DEFINED INSIDE THE CLASS

```
class item
{
    int    number;
    float cost;

public:
    void getdata(int a, float b);    // declaration only

    void putdata(void)               // definition inside
    {
        cout << number << endl;
        cout << cost   << endl;
    }
};
```

Normally only small functions are defined inside the class definition — which fits perfectly with what you learned about inline functions in Chapter 32.

Making an outside function inline explicitly

You can have it both ways: define the function outside for readability, and still ask for it to be inlined.

● EXPLICIT INLINE MEMBER FUNCTION

```
class item
{
    int    number;
    float cost;
public:
    void getdata(int a, float b);
};

inline void item :: getdata(int a, float b)
{
    number = a;
    cost   = b;
}
```

Nesting of member functions

A member function can be called by **another member function of the same class**. This is known as **nesting of member functions** — and no dot operator is needed.

● ONE MEMBER FUNCTION CALLING ANOTHER

```
class sample
{
    int m;
    void read(void);    // PRIVATE member function

public:
    void update(void);
    void write(void);
};

void sample :: update(void)
{
    read();            // calling a private member function
}
```

Private member functions

Data is normally private, but a **function** can be private too. You would do this for a helper function that should never be called from outside — an internal detail of how the class works.

● WHAT AN OBJECT MAY NOT DO

```
sample S1;
S1.read();    // WON'T WORK — read() is private
```

WORTH REMEMBERING

A private member function can only be called by another function that is a member of its class. Even an object of that same class cannot call it directly. The only way in is through a public function like `update()`, which then calls `read()` internally.

CHAPTER 38

Arrays inside a class

An array can be used as a data member of a class, exactly like any other variable. The array is then private, so only the class's own functions can read or write its elements.

● AN ARRAY AS A CLASS MEMBER

```
const int size = 50;

class array
{
    int arr[size];    // private array

public:
    void setval(void);
    void display(void);
};
```

The array `arr` contains 50 integers, and it is private. Only `setval()` and `display()` can get at it, so no code outside the class can accidentally write past the end of the array or corrupt its contents.

● A WORKED EXAMPLE – A SIMPLE MARKS STORE

```
#include <iostream>
using namespace std;

const int size = 5;

class marks
{
    int arr[size];
    int count;

public:
    void setval(void)
    {
        count = 0;
        for (int i = 0; i < size; i++)
        {
            cout << "Enter mark " << i + 1 << ": ";
            cin >> arr[i];
            count++;
        }
    }

    void display(void)
    {
        int total = 0;
        for (int i = 0; i < count; i++)
        {
            cout << "Mark " << i + 1 << " = " << arr[i] << endl;
            total = total + arr[i];
        }
        cout << "Total = " << total << endl;
    }
};

int main()
{
    marks m1;
    m1.setval();
    m1.display();
    return 0;
}
```

IN PLAIN WORDS

Notice that `count` and `arr` travel together. Outside code can never set `count` to a wrong value and make `display()` read past the end of the array. That safety is exactly what you were promised in Chapter 09 — and here it is, actually happening.

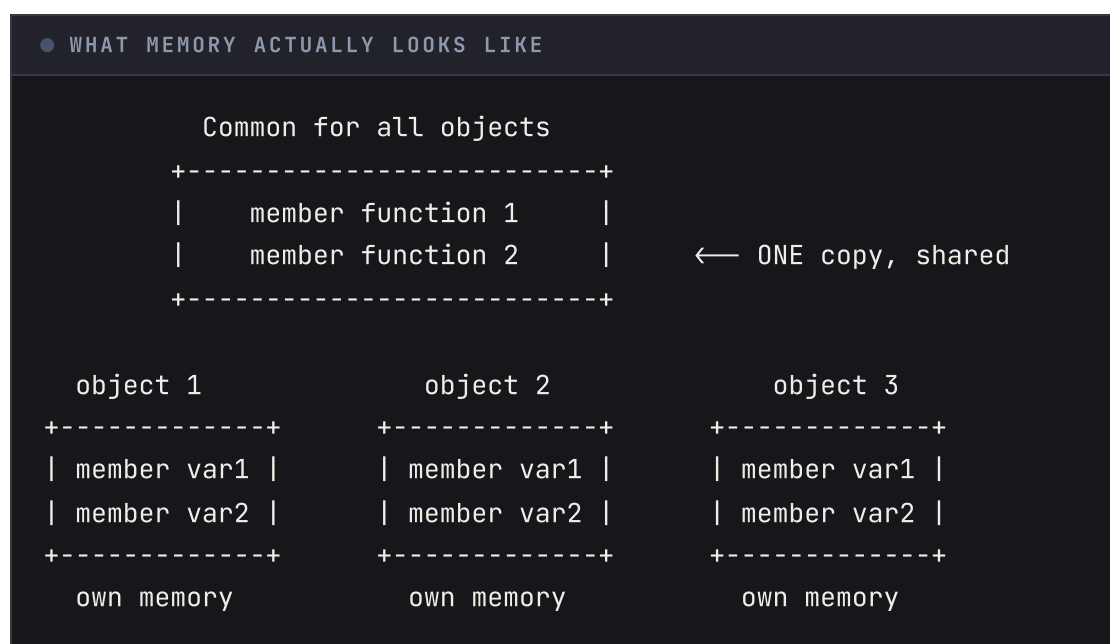
CHAPTER 39

Memory allocation for objects

This question is asked constantly: if I make ten objects of a class, do I get ten copies of the member functions too? The answer is no — and understanding why makes the rest of the course easier.

The rule

- **Member functions are created and placed in memory only once**, when the class is defined.
- All objects belonging to that class **share** that one copy of each member function.
- **Space for member variables is allocated separately for each object**, because the member variables hold different values in different objects.



IN PLAIN WORDS

The recipe for making tea is written once in the cookbook. A hundred people can follow it. But each person needs their own cup. The recipe is the member function; the cup is the member variable.

WORTH REMEMBERING

Memory space for member variables is allocated separately for each object; member functions are stored once and shared by all objects of the class. That single sentence is a complete answer to a very common exam question.

Arrays of objects

Since a class is a data type, you can make an array of objects just as you would make an array of `int`.

● ARRAYS OF EMPLOYEE OBJECTS

```
employee manager[3];    // array of 3 manager objects
employee developer[15];  // array of 15 developer objects
employee tester[30];     // array of 30 tester objects
```

The array `manager` contains three objects, named `manager[0]`, `manager[1]` and `manager[2]`, each of type `employee`. You call member functions on them with the dot operator as usual:

● USING AN ARRAY OF OBJECTS

```
for (int i = 0; i < 3; i++)
{
    manager[i].getdata();
}

for (int i = 0; i < 3; i++)
{
    manager[i].putdata();
}
```

CHAPTER 40

static data members and member functions

Normally every object gets its own copy of every data member. A `static` member breaks that rule on purpose: **one copy, shared by every object of the class.**

static data members

THE FOUR PROPERTIES

- It is **initialised to zero** when the first object of its class is created. No other initialisation is permitted.
- **Only one copy** of that member is created for the entire class and is **shared by all the objects** of that class, no matter how many objects are created.
- It is **visible only within the class**, but its lifetime is the entire program.
- It must be **defined outside the class** to actually allocate storage for it.

● DEFINING IT OUTSIDE

```
type class-name :: member-name = value;
```

● REAL EXAMPLE

```
int item :: count = 10;
```

A complete counter

● `STATICCOUNT.CPP`

```
#include <iostream>
using namespace std;

class item
{
    static int count;    // declared, NOT defined
    int number;

public:
    void getdata(int a)
    {
        number = a;
        count++;        // shared across all objects
    }

    void getcount(void)
    {
        cout << "count: " << count << endl;
    }
};

int item :: count;      // definition – storage happens here

int main()
{
    item a, b, c;        // three objects

    a.getcount();        // count is 0
    b.getcount();
    c.getcount();

    a.getdata(100);
    b.getdata(200);
    c.getdata(300);

    cout << "After reading data" << endl;

    a.getcount();        // all three now show 3
    b.getcount();
    c.getcount();

    return 0;
}
```

OUTPUT

```
count: 0
count: 0
count: 0
After reading data
count: 3
count: 3
count: 3
```

This is the proof. Objects a, b and c each have their own `number`, but they all read the **same** `count`. Each call to `getdata()` incremented the one shared copy.

CAREFUL

The line `int item :: count;` outside the class is **not optional**. Inside the class you only *declare* that a static member exists. If you leave out the outside definition, the program compiles but fails at link time with "undefined reference to `item::count`".

static member functions

A member function can also be declared `static`. Such a function has two restrictions:

- A static member function can **have access only to other static members** — functions or variables — declared in the same class.
- A static member function can be **called using the class name instead of an object**.

● GENERAL FORM OF THE CALL

```
class-name :: function-name();
```

● REAL EXAMPLE

```
item :: getcount();    // no object needed
```

IN PLAIN WORDS — WHY THE RESTRICTION MAKES SENSE

A static function can be called without any object existing. So if it tried to read `number`, the compiler would have to ask: *whose number?* There is no object to ask. That is exactly why a static function may only touch static data — static data belongs to the class, not to any object.

WORTH REMEMBERING

A perfect use case for a static member: **counting how many objects of a class currently exist**. No single object can know this, because it is a fact about the class as a whole. That is the standard exam example, and it is the program above.

CHAPTER 41

Objects as arguments and return values

An object is a data type like any other, so it can be passed into a function and handed back out of one.

Passing an object to a function

An object may be used as a function argument in two ways:

- A copy of the entire object is passed to the function — this is **pass by value**. Since a copy is made, changes made to the object inside the function do not affect the object used in the call.
- Only the address of the object is transferred to the function — this is **pass by reference**. Here, changes made to the object inside the function *will* affect the original object, since they are on the same object.

● OBJECT PASSED BY VALUE

```
void calculateAverage(Student s1, Student s2)
{
    // s1 and s2 are COPIES
}
```

● BY VALUE – ORIGINAL SAFE

```
void raise(Student s)
{
    s.marks = s.marks + 10;
}
// caller's object unchanged
```

● BY REFERENCE – ORIGINAL CHANGED

```
void raise(Student &s)
{
    s.marks = s.marks + 10;
}
// caller's object IS changed
```

Returning an object from a function

A function can also hand an object back.

● RETURNING AN OBJECT

```
Student createStudent()
{
    Student student;
    // fill in the student here
    return student;
}
```

A complete example — adding two complex numbers

This program uses both ideas at once: an object goes in as an argument, and a fresh object comes back as the result.

● COMPLEXADD.CPP

```
#include <iostream>
using namespace std;

class complex
{
    float x;    // real part
    float y;    // imaginary part

public:
    void input(float real, float imag)
    {
        x = real;
        y = imag;
    }

    // takes an object IN, hands an object BACK
    complex sum(complex c)
    {
        complex temp;
        temp.x = x + c.x;    // x is this object's; c.x is the argu
        temp.y = y + c.y;
        return temp;
    }

    void show(void)
    {
        cout << x << " + j" << y << endl;
    }
};

int main()
{
    complex c1, c2, c3;

    c1.input(2.5, 3.5);
    c2.input(1.6, 2.7);

    c3 = c1.sum(c2);    // c1 is the caller, c2 is the argument

    cout << "c1 = "; c1.show();
    cout << "c2 = "; c2.show();
    cout << "c3 = "; c3.show();

    return 0;
}
```

OUTPUT

```
c1 = 2.5 + j3.5
c2 = 1.6 + j2.7
c3 = 4.1 + j6.2
```

WORTH REMEMBERING — THE ONE SURPRISING RULE

Inside `sum()`, the line `temp.x = x + c.x;` reads `c.x` — a **private** member of a different object. Is that allowed? **Yes**. Privacy in C++ is *per class*, not per object. A member function of `complex` can reach the private members of **any** `complex` object, not just its own.

IN PLAIN WORDS

Remember this program. In Chapter 48 you will rewrite the exact same thing so that you can simply write `c3 = c1 + c2;` instead of `c3 = c1.sum(c2);`. That is what operator overloading buys you, and comparing the two versions is the fastest way to understand it.

PART VII

Friends and pointers to members

Two escape hatches. The first lets a chosen outsider reach inside a class. The second lets you point at a member of a class the way you point at a variable.

CHAPTER 42

Friend functions

We spent all of Part VI learning that private data is private. A **friend function** is the deliberate exception — a non-member function that the class explicitly grants access to.

What a friend function is

- A friend function is a **function that is not a member of a class but has access to the class's private and protected members**.
- Friend functions are **not considered class members**; they are ordinary external functions that are given special access privileges.
- Friend functions are **not in the class's scope**, and they are **not called using the dot operator**. A friend function can be called like a normal function, without any object.
- A friend function **cannot access member names directly** — it has to use an object name and the dot operator with each member name, like `A.x`.

The syntax

● GENERAL FORM

```
class className
{
    ... ..
    friend returnType functionName(arguments);
    ... ..
};
```

● REAL EXAMPLE

```
class MyClass
{
    int member1;
    friend int init (MyClass);
};

int init(MyClass obj)
{
    obj.member1 = 5;    // allowed – it is a friend
    return obj.member1;
}
```

CAREFUL — WHERE THE KEYWORD GOES

The word `friend` appears **only inside the class declaration**. When you write the actual function definition later, you do **not** repeat `friend`, and you do **not** write `MyClass::` — because it is not a member of the class.

A complete working example

● FRIENDFN.CPP

```
#include <iostream>
using namespace std;

class sample
{
    int a;
    int b;

public:
    void setvalue()
    {
        a = 25;
        b = 40;
    }

    friend float mean(sample s);    // declared a friend
};

float mean(sample s)                // no "friend", no "sample::"
{
    return float(s.a + s.b) / 2.0; // reaches private a and b
}

int main()
{
    sample X;
    X.setvalue();

    cout << "Mean value = " << mean(X) << endl; // called like a n

    return 0;
}
```

OUTPUT

Mean value = 32.5

Three things to notice, and they are all examinable:

1. `mean` reads `s.a` and `s.b`, which are **private**. Only the `friend` declaration makes this legal.
2. The call is `mean(X)`, **not** `X.mean()`. It is not a member, so there is no dot.

3. Inside, it must write `s.a` — it cannot write bare `a`, because it has no object of its own.

Making a member function of another class a friend

You can be more precise than granting friendship to a free function: you can name a **single member function of another class** as the friend.

```
● ONE MEMBER FUNCTION AS FRIEND

class X
{
    ...
    int fun1();
    ...
};

class Y
{
    ...
    friend int X :: fun1();    // only fun1 of class X is a friend
    ...
};
```

The function `fun1()` of class X is a friend of class Y. **No other function of X gets access.** This is the tightest form of friendship and is the one to prefer.

Friend classes

If you want **all** the member functions of one class to have access to another class, declare the whole class a friend.

```
● A WHOLE CLASS AS FRIEND

class Z
{
    ...
    friend class X;    // every member function of X is now a friend
    ...
};
```

Forward declaration

Sometimes a class must be mentioned before it has been fully defined — for example when two classes refer to each other. For this you use a **forward declaration**: you tell the compiler "a class with this name exists, details to follow".

● FORWARD DECLARATION

```
class ABC;    // forward declaration – just the name
```

Where friends are genuinely useful

The real reason friends exist is in **operator overloading**, and you will meet it properly in Chapter 49. The short version: a member operator function always takes its left operand from the object it belongs to. So `A = B + 2;` works, but `A = 2 + B;` fails — because 2 is not an object and has no member functions. A friend function takes **both** operands as explicit arguments, so both orders work.

WORTH REMEMBERING

The standard exam framing is: **"A friend function breaks encapsulation, so why does C++ allow it?"** Answer: because some operations are genuinely symmetric between two classes, or between a class and a built-in type, and forcing them to be a member of one side would be artificial. Friendship is *granted by the class itself*, in its own declaration — so the class still controls who gets in. It is a controlled exception, not a hole.

CAREFUL — THREE PROPERTIES OF FRIENDSHIP

- **Friendship is granted, not taken.** A function cannot declare itself a friend of a class.
- **Friendship is not mutual.** If A declares B a friend, that does not make A a friend of B.
- **Friendship is not inherited.** A friend of a base class is not automatically a friend of a derived class.

CHAPTER 43

const member functions

If a member function does not modify the object it belongs to, you can — and should — say so by marking it `const`. The compiler then enforces the promise.

● GENERAL FORM

```
return_type class_name :: function_name() const
{
    // body — may not modify any data member
}
```

● DECLARATION INSIDE THE CLASS

```
class item
{
    int number;
public:
    int getdata() const;    // const goes at the END
};
```

The `const` keyword goes **after** the argument list, before the body. Both the declaration and the definition must carry it.

● A CONST MEMBER FUNCTION IN FULL

```
#include <iostream>
using namespace std;

class item
{
    int number;
    float cost;

public:
    void setdata(int n, float c)
    {
        number = n;
        cost = c;
    }

    int getnumber() const           // promises not to change anything
    {
        return number;
    }

    void show() const
    {
        cout << number << " costs " << cost << endl;
    }
};

int main()
{
    item i1;
    i1.setdata(101, 45.5);
    cout << "Number is " << i1.getnumber() << endl;
    i1.show();
    return 0;
}
```

OUTPUT

```
Number is 101
101 costs 45.5
```

CAREFUL — THE COMPILER WILL STOP YOU

If you write `number = 5;` inside a function marked `const`, the program will **not compile**. That is the point. The `const` is a promise checked at compile time, not a comment.

WORTH REMEMBERING

Mark every "getter" or display function `const`. Two reasons:

- It documents your intent — the reader knows at a glance that this function is safe.
- A `const` object can **only** call `const` member functions. If you forget the `const`, your function becomes unusable on `const` objects and on `const` references.

CHAPTER 44

Pointers to members

C++ permits us to define a class containing various types of data and functions as members. It also permits us to access those class members *through pointers*. To do this, C++ provides a set of three pointer-to-member operators.

OPERATOR	WHAT IT DOES
<code>::*</code>	To declare a pointer to a member of a class.
<code>.*</code>	To access a member using an object name and a pointer to that member.
<code>→*</code>	To access a member using a pointer to the object and a pointer to that member.

First, the ordinary pointer to an object

Before the new operators, remember the plain case. A pointer can point at an object, and then you reach members with `→`:

● POINTER TO AN OBJECT

```
class Simple
{
public:
    int a;
};

int main()
{
    Simple obj;
    obj.a = 10;

    Simple* ptr;
    ptr = &obj;

    cout << ptr->a;    // 10
    return 0;
}
```

Pointer to a data member

This is different. Here the pointer does not point at an object — it points at a *member slot* that exists in every object of the class.

● GENERAL FORM

```
datatype class_name :: *pointer_name;

pointer_name = &class_name :: datamember_name;
```

● REAL EXAMPLE

```
int A :: *ip = &A :: m;
```

The line `int A :: *ip = &A :: m;` reads as: "*ip is a pointer to an int member of class A, and it points at the member called m.*" Note that `m` must be **public** for this to be written outside the class.

Using it with . * and ->*

- WITH AN OBJECT – USE .*

```
A a;  
cout << a.*ip;
```

- WITH A POINTER TO OBJECT – USE ->*

```
A *ap = &a;  
cout << ap->*ip;
```

A complete program

● PTRMEMBER.CPP

```
#include <iostream>
using namespace std;

class Data
{
public:
    int a;

    void print()
    {
        cout << "a = " << a << endl;
    }
};

int main()
{
    Data d;
    Data *dp = &d;

    int Data::*ptr = &Data::a;    // pointer to the member 'a'

    d.*ptr = 10;                   // set through an object
    d.print();                     // a = 10

    dp->*ptr = 20;                  // set through a pointer to object
    dp->print();                   // a = 20

    return 0;
}
```

OUTPUT

```
a = 10
a = 20
```

Pointer to a member function

The same idea applied to functions. The syntax looks heavier because the pointer must also record the return type and the argument types.

● DECLARE AND ASSIGN TOGETHER

```
int (Data::*fp1) (float) = &Data::f;
```

● DECLARE, THEN ASSIGN

```
int (Data::*fp2) (float);  
fp2 = &Data::f;
```

Read `int (Data::*fp1)(float)` from the inside out: "*fp1 is a pointer to a member function of Data that takes a float and returns an int.*"

● CALLING THROUGH A MEMBER FUNCTION POINTER

```
#include <iostream>  
using namespace std;  
  
class Data  
{  
public:  
    int f(float x)  
    {  
        cout << "f called with " << x << endl;  
        return 1;  
    }  
};  
  
int main()  
{  
    Data d;  
    Data *dp = &d;  
  
    int (Data::*fp)(float) = &Data::f;  
  
    (d.*fp)(3.5);           // through an object  
    (dp->*fp)(7.5);         // through a pointer to object  
  
    return 0;  
}
```

OUTPUT

```
f called with 3.5  
f called with 7.5
```

CAREFUL — THE BRACKETS ARE COMPULSORY

You must write `(d.*fp)(3.5);` with the outer brackets. Without them, `d.*fp(3.5)` is parsed as `d.*(fp(3.5))` — C++ gives the function-call brackets higher precedence than `.*`, so it tries to call `fp` first. This is a guaranteed compile error and a favourite trick question.

IN PLAIN WORDS

A normal pointer stores an *address*. A pointer to a member stores an *offset* — "the third slot in whatever object you give me". That is why you always need two things to use it: the pointer-to-member **and** an object to apply it to. That is exactly what `.*` and `→*` do — they join the two halves together.

PART VIII

The life of an object

How an object is born, how it dies, and how you teach ordinary operators like `+` to work on it.

CHAPTER 45

Constructors

A **constructor** is a special member function whose task is to **initialise the objects of its class**. It is called automatically the moment an object is created — you never call it yourself.

The four rules

A CONSTRUCTOR MUST OBEY ALL FOUR

- It has **the same name as the class** itself.
- It is **called automatically** when an object is created.
- It has **no return type** — not even void.
- It should be declared in the **public** section of the class.

● GENERAL FORM

```
class Wall
{
public:
    Wall()          // same name as class
    {
        // body of constructor
    }
};
```

● REAL EXAMPLE

```
class integer
{
    int m, n;
public:
    integer(void);    // constructor declared
};

integer :: integer(void)    // defined
{
    m = 0;
    n = 0;
}
```

When you now write:

● CREATING THE OBJECT

```
integer int1;    // constructor runs automatically, m and n become 0
```

...the declaration not only creates the object `int1` of type `integer` but also **initialises its data members `m` and `n` to zero**.

IN PLAIN WORDS — WHY CONSTRUCTORS EXIST

Without a constructor, a freshly created object contains whatever rubbish was already in that memory. A constructor guarantees that **an object is never in an invalid state, not even for a single instruction**. That is a big deal. It means you can never accidentally use an uninitialised object.

Default constructor

A constructor that **takes no arguments** is called the **default constructor**.

● A DEFAULT CONSTRUCTOR WITH A BODY

```
#include <iostream>
using namespace std;

class Wall
{
private:
    double length;

public:
    Wall()
    {
        length = 5.5;
        cout << "Creating a wall." << endl;
        cout << "Length = " << length << endl;
    }
};

int main()
{
    Wall wall1;    // constructor runs here
    return 0;
}
```

OUTPUT

```
Creating a wall.
Length = 5.5
```

Parameterized constructor

Often you want to initialise different objects with **different values**. C++ permits you to pass arguments to the constructor. Such constructors are called **parameterized constructors**.

● A PARAMETERIZED CONSTRUCTOR

```
class integer
{
    int m, n;

public:
    integer(int x, int y);    // parameterized constructor
};

integer :: integer(int x, int y)
{
    m = x;
    n = y;
}
```

Now you must supply the values when you create the object. There are two ways to do that.

● EXPLICIT CALL

```
integer int1 = integer(0, 10);
```

● IMPLICIT CALL (SHORTHAND)

```
integer int2(100, 500);
```

Both do exactly the same thing. The second form is shorter and is the one you will normally see.

CAREFUL

Once you write **any** constructor of your own, C++ stops providing the **free default one**. So if you only write `integer(int x, int y)`, the line `integer a;` becomes a compile error. If you need both forms, write both constructors.

Constructor overloading

Constructors are functions, so they can be overloaded just like any other function. This is called **multiple constructors in a class**.

● THREE CONSTRUCTORS IN ONE CLASS

```
class integer
{
    int m, n;

public:
    integer()                // 1. default constructor
    {
        m = 0;
        n = 0;
    }

    integer(int a, int b)    // 2. parameterized constructor
    {
        m = a;
        n = b;
    }

    integer(integer &i)      // 3. copy constructor
    {
        m = i.m;
        n = i.n;
    }
};

int main()
{
    integer I1;              // calls constructor 1 → m=0, n=0
    integer I2(20, 40);      // calls constructor 2 → m=20, n=40
    integer I3(I2);          // calls constructor 3 → copies I2
    return 0;
}
```

Constructors with default arguments

Everything you learned in Chapter 33 applies here too.

● DEFAULT ARGUMENT IN A CONSTRUCTOR

```
class complex
{
    float real, imag;
public:
    complex(float r, float i = 0);    // i defaults to 0
};

complex :: complex(float r, float i)
{
    real = r;
    imag = i;
}

int main()
{
    complex c1(5.0);                // real = 5.0, imag = 0
    complex c2(5.0, 3.5);           // real = 5.0, imag = 3.5
}
```

CAREFUL — A HIDDEN AMBIGUITY

If a class has both `complex()` and `complex(float r = 0, float i = 0)`, then the statement `complex c;` is **ambiguous** — the compiler cannot tell which one you meant. Both can be called with zero arguments. Do not write both.

Dynamic initialisation of objects

Remember dynamic initialisation from Chapter 20? Class objects can be initialised dynamically too — the values passed to a constructor can come from anywhere, including the user at run time.

● VALUES DECIDED AT RUN TIME

```
int main()
{
    float p, r;
    int t;

    cout << "Enter principal, rate and time: ";
    cin >> p >> r >> t;

    account a1(p, r, t);    // constructor gets run-time values

    return 0;
}
```

The advantage: you can provide various initialisation formats, using overloaded constructors, and choose between them while the program is running.

Dynamic constructors

A **dynamic constructor** is one that allocates memory with `new` while the object is being built. This lets an object decide *at run time* exactly how much memory it needs.

● A CONSTRUCTOR THAT ALLOCATES MEMORY

```
#include <iostream>
#include <cstring>
using namespace std;

class String
{
    char *name;
    int length;

public:
    String(char *s)           // dynamic constructor
    {
        length = strlen(s);
        name = new char[length + 1]; // memory taken HERE
        strcpy(name, s);
    }

    void display(void)
    {
        cout << name << endl;
    }
};

int main()
{
    String s1("SVNIT");
    String s2("Surat");

    s1.display();
    s2.display();

    return 0;
}
```

OUTPUT

SVNIT
Surat

WORTH REMEMBERING

Note the `length + 1`. The extra byte is for the terminating `\0` — exactly the rule you met in Chapter 16. Note too that this class **takes memory but never gives it back**. That is the problem the next chapter but one solves.

CHAPTER 46

The copy constructor

A **copy constructor** is used to declare and initialise an object from another object of the same class. It has one rule that trips up almost everybody.

The rule that matters

● ILLEGAL

```
class A
{
public:
    A(A);    // ERROR – by value
};
```

● LEGAL – THIS IS A COPY CONSTRUCTOR

```
class A
{
public:
    A(A&);    // by reference
};
```

WORTH REMEMBERING — WHY A(A) IS ILLEGAL

Passing an object **by value** means making a copy of it. Making a copy means calling the copy constructor. So `A(A)` would have to call itself to build its own argument, which would call itself again, forever. **The copy constructor must take its argument by reference.** This is asked constantly.

Writing one

● COPY CONSTRUCTOR IN A CLASS

```
class integer
{
    int m, n;

public:
    integer()
    {
        m = 0;
        n = 0;
    }

    integer(int a, int b)
    {
        m = a;
        n = b;
    }

    integer(integer &i)    // the copy constructor
    {
        m = i.m;
        n = i.n;
    }
};
```

The four ways it gets called

● WHERE A COPY CONSTRUCTOR RUNS

```
integer I2(20, 40);

integer I3(I2);    // 1. explicit copy – the common form
integer I4 = I2;   // 2. copy initialization – same thing
```

Beyond those two, the copy constructor also runs invisibly:

1. When an object is **passed to a function by value** — the function's copy is built by the copy constructor.
2. When an object is **returned from a function by value**.

CAREFUL — THE DISTINCTION PEOPLE GET WRONG

`integer I4 = I2;` calls the **copy constructor**, because I4 is being *created*. But `I4 = I2;` on a later line, when I4 already exists, calls the **assignment operator**, not the copy constructor. Same symbol, two different jobs, decided by whether the object already exists.

A complete demonstration

● COPYCTOR.CPP

```
#include <iostream>
using namespace std;

class code
{
    int id;

public:
    code()                { }                // default
    code(int a)           { id = a; }         // parameterized
    code(code &x)          { id = x.id; }      // copy constructor

    void display(void)
    {
        cout << id;
    }
};

int main()
{
    code A(100);          // parameterized constructor
    code B(A);            // copy constructor — explicit
    code C = A;           // copy constructor — copy initialization
    code D;
    D = A;                // assignment, NOT the copy constructor

    cout << "\n id of A: "; A.display();
    cout << "\n id of B: "; B.display();
    cout << "\n id of C: "; C.display();
    cout << "\n id of D: "; D.display();

    return 0;
}
```

OUTPUT

```
id of A: 100  
id of B: 100  
id of C: 100  
id of D: 100
```

CHAPTER 47

Destructors

A **destructor** is the opposite of a constructor. It is called automatically when an object is destroyed, and its job is to clean up.

The rules

A DESTRUCTOR

- Has the same name as the class, preceded by a tilde ~.
- Takes **no arguments** and returns **no value**.
- Is called **automatically** when the object goes out of scope.
- There can be **only one destructor** per class — it cannot be overloaded.

● GENERAL FORM

```
~ class-name()
{
    // clean-up code
}
```

● REAL EXAMPLE

```
~ integer()
{
}
```

When it actually runs

● CTORDTOR.CPP

```
#include <iostream>
using namespace std;

int count = 0;

class alpha
{
public:
    alpha()                // constructor
    {
        count++;
        cout << "No. of object created: " << count << endl;
    }

    ~alpha()               // destructor
    {
        cout << "No. of object destroyed: " << count << endl;
        count--;
    }
};

int main()
{
    cout << "\n\nENTER MAIN\n";
    alpha A1, A2, A3;
    {
        cout << "\nENTER BLOCK1\n";
        alpha A4;
    }                    // A4 destroyed HERE
    cout << "\nRE-ENTER MAIN\n";
    return 0;
}                        // A3, A2, A1 destroyed HERE
```

OUTPUT

```
ENTER MAIN
No. of object created: 1
No. of object created: 2
No. of object created: 3

ENTER BLOCK1
No. of object created: 4
No. of object destroyed: 4

RE-ENTER MAIN
No. of object destroyed: 3
No. of object destroyed: 2
No. of object destroyed: 1
```

WORTH REMEMBERING — THE ORDER

Objects are destroyed in the reverse order of their creation. A1, A2, A3 were created in that order, so they die A3, A2, A1. Notice also that A4 died at the closing brace of its inner block, before main had finished. Destruction follows scope, exactly.

The real job of a destructor: releasing memory

Remember the `String` class from Chapter 45 that called `new` and never called `delete`? Here is the fix.

● CONSTRUCTOR TAKES, DESTRUCTOR GIVES BACK

```
#include <iostream>
#include <cstring>
using namespace std;

class String
{
    char *name;
    int length;

public:
    String(char *s)                // constructor
    {
        length = strlen(s);
        name = new char[length + 1]; // TAKE memory
        strcpy(name, s);
        cout << "Allocated for: " << name << endl;
    }

    ~String()                      // destructor
    {
        cout << "Releasing: " << name << endl;
        delete [] name;           // GIVE IT BACK
    }

    void display(void)
    {
        cout << name << endl;
    }
};

int main()
{
    String s1("SVNIT");
    s1.display();
    return 0;

    // destructor runs here
}
```

OUTPUT

```
Allocated for: SVNIT
SVNIT
Releasing: SVNIT
```

WORTH REMEMBERING — THE GOLDEN PAIRING

Whatever the constructor allocates with `new`, the destructor must release with `delete`. Constructor takes, destructor gives back. Get into this habit now and you will never write a memory leak.

And note the form: `delete [] name;` with brackets, because `new char[ ... ]` allocated an array.

Constructor and destructor side by side

	CONSTRUCTOR	DESTRUCTOR
Name	Same as the class	Same as the class with <code>~</code> in front
When it runs	When an object is created	When an object is destroyed
Arguments	Can take arguments	Never takes arguments
Return type	None	None
Overloading	Can be overloaded	Cannot be overloaded — only one per class
Typical job	Initialise data, allocate memory	Release memory, close files
Order for several objects	Order of creation	Reverse order of creation

CHAPTER 48

Operator overloading

C++ lets you give an existing operator like `+` an additional meaning when it is applied to your own class. This is the second form of polymorphism, and it is what makes C++ classes feel like built-in types.

The idea

In Chapter 41 you wrote `c3 = c1.sum(c2);` to add two complex numbers. That works, but it reads badly. What you actually want is:

● THE GOAL

```
c3 = c1 + c2;
```

Operator overloading is how you get there. To define an additional task for an operator, you must specify what it means **in relation to the class to which the operator is applied**. This is done with a special function called an **operator function**.

The syntax

● GENERAL FORM – OUTSIDE THE CLASS

```
return-type classname :: operator op(arglist)
{
    Function body
}
```

● GENERAL FORM – INSIDE THE CLASS

```
class className
{
public:
    returnType operator symbol (arguments)
    {
        // body
    }
};
```

Three things make up the function name: the keyword `operator`, the operator symbol itself, and the argument list.

Adding two complex numbers

● OPERATORPLUS.CPP

```
#include <iostream>
using namespace std;

class Complex
{
private:
    float real, imag;

public:
    void input(float r, float i)
    {
        real = r;
        imag = i;
    }

    // overloading the + operator
    Complex operator + (const Complex& obj)
    {
        Complex temp;
        temp.real = real + obj.real;
        temp.imag = imag + obj.imag;
        return temp;
    }

    void show()
    {
        cout << real << " + j" << imag << endl;
    }
};

int main()
{
    Complex complex1, complex2, result;

    complex1.input(2.5, 3.5);
    complex2.input(1.6, 2.7);

    result = complex1 + complex2;    // the overloaded + runs here

    cout << "Result = ";
    result.show();

    return 0;
}
```

OUTPUT

```
Result = 4.1 + j6.2
```

What the compiler really does

This is the single most useful thing to understand about operator overloading.

● WHAT YOU WRITE

```
result = complex1 + complex2;
```

● WHAT THE COMPILER TURNS IT INTO

```
result = complex1.operator+(complex2);
```

WORTH REMEMBERING — THE TRANSLATION RULE

For a **member** operator function, `a + b` becomes `a.operator+(b)`. The **left operand becomes the calling object**, and the **right operand becomes the argument**. Once you see this, everything else about operator overloading follows — including why some things do not work, which is Chapter 49.

Unary operator overloading

A **unary** operator works on one operand — like `-` for negation, or `++` for increment. As a member function, a unary operator takes **no arguments**, because the single operand is the calling object itself.

● OVERLOADING UNARY MINUS

```
#include <iostream>
using namespace std;

class space
{
    int x, y, z;

public:
    void getdata(int a, int b, int c)
    {
        x = a; y = b; z = c;
    }

    void display(void)
    {
        cout << x << " " << y << " " << z << endl;
    }

    void operator-()           // unary minus - NO arguments
    {
        x = -x;
        y = -y;
        z = -z;
    }
};

int main()
{
    space S;
    S.getdata(10, -20, 30);

    cout << "S : ";  S.display();

    -S;               // calls S.operator-()

    cout << "-S: ";  S.display();

    return 0;
}
```

OUTPUT

```
S : 10 -20 30
-S: -10 20 -30
```

Prefix and postfix ++

The increment operator has two forms, `++x` and `x++`, and C++ needs a way to tell them apart. The trick is a dummy `int` argument that marks the postfix form.

● PREFIX – ++X

```
void operator ++ ()
{
    // prefix form – no argument
}
```

● POSTFIX – X++

```
void operator ++ (int)
{
    // postfix form – dummy int
}
```

IN PLAIN WORDS

The `int` in `operator ++ (int)` is never used and has no name. It exists purely as a signal to the compiler meaning "this is the postfix version". It is a slightly odd piece of syntax, but it is the standard way, and it is examinable.

The rules for operator overloading

THE SEVEN RULES

- **Only existing operators can be overloaded.** You cannot invent a new symbol.
- The overloaded operator must have **at least one operand that is a user-defined type**.
- You **cannot change the basic meaning** of an operator — you cannot make `+` mean subtraction for integers.
- Overloaded operators **follow the syntax rules of the original operators**. Their precedence and associativity cannot be changed.
- The **number of operands cannot be changed**. A unary operator stays unary; a binary operator stays binary.
- Some operators **cannot be overloaded at all** — see the list below.
- Some operators **cannot be overloaded using a friend function** — see Chapter 49.

The four operators that can never be overloaded

OPERATOR	NAME	WHY NOT
::	Scope resolution operator	It works on names, not on values.
.	Member access operator	Overloading it would make member access itself ambiguous.
.*	Pointer-to-member access operator	Same reason as above.
?:	Conditional (ternary) operator	It evaluates only one branch; a function call would evaluate both.

WORTH REMEMBERING

Memorise these four: `::` `.` `.*` `?:`. "Which operators cannot be overloaded in C++?" is a guaranteed question, and this is the complete answer.

CHAPTER 49

Member function or friend function?

An operator can be overloaded in two ways: as a **member function** of the class, or as a **friend function**. They are not interchangeable, and knowing when each one works is the point of this chapter.

The two translations

● MEMBER FUNCTION

```
C3 = C1 + C2;  
  
// becomes  
C3 = C1.operator+(C2);  
  
// left operand = calling object  
// right operand = argument  
// takes ONE argument
```

● FRIEND FUNCTION

```
C3 = C1 + C2;  
  
// becomes  
C3 = operator+(C1, C2);  
  
// BOTH operands are arguments  
// takes TWO arguments
```

The situation where a member function fails

Suppose you want to add a plain number to an object. With a member function:

● THIS WORKS

```
A = B + 2;  
  
// becomes B.operator+(2)  
// B is an object → it has member  
// functions → fine
```

● THIS FAILS

```
A = 2 + B;  
  
// would become 2.operator+(B)  
// 2 is an int. An int has no  
// member functions. ERROR.
```

WORTH REMEMBERING — THE KEY SENTENCE

`A = 2 + B;` fails with a member function but works with a friend function. With a friend, it simply becomes `operator+(2, B)` — an ordinary two-argument function call, and there is no requirement that the first argument be an object. This is exactly why friend functions exist, and it is the most commonly asked question on this topic.

Writing it both ways

● FRIEND VERSION – HANDLES BOTH ORDERS

```
#include <iostream>
using namespace std;

class Complex
{
    float real, imag;

public:
    void input(float r, float i)
    {
        real = r;
        imag = i;
    }

    // two friends, one for each order
    friend Complex operator + (Complex c, float n);
    friend Complex operator + (float n, Complex c);

    void show()
    {
        cout << real << " + j" << imag << endl;
    }
};

Complex operator + (Complex c, float n)    // object + number
{
    Complex temp;
    temp.real = c.real + n;
    temp.imag = c.imag;
    return temp;
}

Complex operator + (float n, Complex c)    // number + object
{
    Complex temp;
    temp.real = n + c.real;
    temp.imag = c.imag;
    return temp;
}

int main()
{
    Complex B, A1, A2;
    B.input(3.0, 4.0);

    A1 = B + 2.0;    // works
```

```

    A2 = 2.0 + B;           // ALSO works, thanks to the friend

    cout << "B + 2 = ";  A1.show();
    cout << "2 + B = ";  A2.show();

    return 0;
}

```

OUTPUT

```

B + 2 = 5 + j4
2 + B = 5 + j4

```

How many arguments each form takes

OPERATOR KIND	AS A MEMBER FUNCTION	AS A FRIEND FUNCTION
Unary (one operand)	0 arguments	1 argument
Binary (two operands)	1 argument	2 arguments

The pattern is simple: a friend function always needs **one more argument** than the member version, because it has no calling object to supply the left operand.

Which should you use?

USE A MEMBER FUNCTION WHEN

- The left operand is always an object of your class
- The operator modifies the object — `=`, `+=`, `++`
- The operator must be a member: `=`, `()`, `[]`, `->`

USE A FRIEND FUNCTION WHEN

- The left operand may be a built-in type — `2 + B`
- You want both operand orders to work
- The operation is symmetric and belongs to neither side

CAREFUL

Four operators **must** be overloaded as member functions and can never be friends: `=` `()` `[]` `→`. Do not confuse this list with the four from Chapter 48 that cannot be overloaded at all (`::`, `.`, `.*`, `?:`). Two different lists, both examinable.

CHAPTER 50

String manipulation using operators

C never let you write `s3 = s1 + s2;` for strings — you had to call `strcat()`. With operator overloading, C++ lets you build a string class that behaves the way strings ought to behave.

The problem with C-style strings

● C STYLE – FUNCTION CALLS

```
char s1[50] = "Hello ";
char s2[50] = "World";

strcat(s1, s2);      // joining
strcmp(s1, s2);      // comparing
strcpy(s1, s2);      // copying
```

● C++ WITH OVERLOADED OPERATORS

```
String s1 = "Hello ";
String s2 = "World";

s3 = s1 + s2;         // joining
if (s1 > s2) ...      // comparing
s1 = s2;              // copying
```

A string class with + overloaded

● STRINGCLASS.CPP

```
#include <iostream>
#include <cstring>
using namespace std;

class String
{
    char *p;
    int len;

public:
    String() // default constructor
    {
        len = 0;
        p = new char[1];
        p[0] = '\\0';
    }

    String(const char *s) // constructor from a C string
    {
        len = strlen(s);
        p = new char[len + 1];
        strcpy(p, s);
    }

    String(const String &s) // copy constructor
    {
        len = s.len;
        p = new char[len + 1];
        strcpy(p, s.p);
    }

    ~String() // destructor
    {
        delete [] p;
    }

    // concatenation
    friend String operator + (const String &a, const String &b);

    // comparison
    friend int operator > (const String &a, const String &b);

    void display(void)
    {
        cout << p << endl;
    }
}
```

```

};

String operator + (const String &a, const String &b)
{
    String temp;
    delete [] temp.p;                // free the empty one

    temp.len = a.len + b.len;
    temp.p   = new char[temp.len + 1];

    strcpy(temp.p, a.p);
    strcat(temp.p, b.p);

    return temp;
}

int operator > (const String &a, const String &b)
{
    return (strcmp(a.p, b.p) > 0) ? 1 : 0;
}

int main()
{
    String s1("Hello, ");
    String s2("World");
    String s3;

    s3 = s1 + s2;                    // the overloaded +

    cout << "s1 = "; s1.display();
    cout << "s2 = "; s2.display();
    cout << "s3 = "; s3.display();

    if (s1 > s2)
        cout << "s1 is bigger" << endl;
    else
        cout << "s2 is bigger" << endl;

    return 0;
}

```

OUTPUT

```

s1 = Hello,
s2 = World
s3 = Hello, World
s2 is bigger

```

Everything in this course, in one program

Look at what that class contains. It is the whole syllabus in forty lines:

FEATURE	WHERE IT APPEARS	CHAPTER
Private data, public functions	<code>char *p;</code> is private	36
Constructor overloading	Three different constructors	45
Dynamic constructor	<code>p = new char[len+1];</code>	45
Copy constructor	<code>String(const String &s)</code>	46
Destructor	<code>~String() { delete [] p; }</code>	47
Friend functions	Both operator functions	42
Operator overloading	<code>operator+</code> and <code>operator></code>	48
const references	<code>const String &a</code>	43

CAREFUL — WHY THE COPY CONSTRUCTOR IS ESSENTIAL HERE

This class holds a raw pointer. Without a copy constructor, copying a `String` would copy the *pointer*, so two objects would point at the same memory. When both destructors ran, that memory would be deleted twice and the program would crash. **Any class that allocates memory with `new` needs its own copy constructor and destructor.** That is not a detail — it is the reason both features exist.

WORTH REMEMBERING

If an exam asks you to "write a program to demonstrate string manipulation using operator overloading", this program is the model answer. The two things the examiner is looking for are a `friend` operator function and a destructor that releases the memory the constructor took.

PART IX

Inheritance: extending classes

The feature the whole of OOP is built around. One class hands its work to another, and the second one only writes what is different.

CHAPTER 51

What inheritance is, and why it exists

Your lecture opens with one word: **reusability**. It is always nicer to reuse something that already exists than to build the same thing again — and a class that has already been tested, debugged and used many times is worth a great deal.

The definition

C++ supports reuse by letting you **create new classes out of existing ones**. The mechanism of deriving a new class from an old one is called **inheritance**, or **derivation**.

Put in the words of the slide: *the capability of a class to derive properties and characteristics from another class is called inheritance*.

THE OLD CLASS	THE NEW CLASS
Base class	Derived class
Parent class	Child class
Super class	Sub class

IN PLAIN WORDS

When we say the derived class *inherits* the base class, we mean it gets all the usable properties of the base class **without changing the base class in any way**, and may then add features of its own. The base class does not know or care that it has been inherited.

The is-a test

Inheritance is an **is-a relationship**. Use it only when that sentence is actually true:

- A car **is a** vehicle.
- An orange **is a** fruit.
- A surgeon **is a** doctor.
- A dog **is an** animal.

CAREFUL

"A car *has an* engine" is **not** an is-a relationship, so Car should not inherit from Engine — it should simply hold an Engine as a member. Getting is-a and has-a the wrong way round is the commonest design mistake in this topic, and examiners ask about it.

Why bother — the three-class problem

Your slide gives the example. You need classes for Bus, Car and Truck. All three need `fuelAmount()`, `capacity()` and `applyBrakes()`, and the code for those three is the same in each.

WITHOUT INHERITANCE

- Write the three functions in Bus
- Write them *again* in Car
- Write them *again* in Truck
- Fix a bug → fix it in three places

WITH INHERITANCE

- Write them once in Vehicle
- Derive Bus, Car, Truck from it
- Each adds only what is different
- Fix a bug → fix it once

That is the whole argument. Inheritance, used to modify and extend the capabilities of existing classes, becomes a very powerful tool for **incremental program development**.

WORTH REMEMBERING

Three benefits to quote: **reusability** (write once, use in many classes), **extensibility** (add to a class without touching it), and **less code to maintain** (one copy to fix). The cost is **tighter coupling** — changing the base class now affects everything derived from it.

CHAPTER 52

Writing a derived class

A derived class is defined by stating its relationship to the base class, and then adding its own details. The syntax is one line of punctuation you must get exactly right.

● GENERAL FORM

```
class derived_name : mode base_name
{
    // members of the derived class
};
```

● REAL EXAMPLE

```
class Car : public Vehicle
{
    public:
        void horn();
};
```

Reading that line piece by piece

class

The keyword that creates a new class.

derived_class_name

The name of the new class, the one doing the inheriting.

:

The colon. It is what says "is derived from". Forget it and you have written two unrelated classes.

visibility mode

private, public or protected. It is **optional**, and if you leave it out the default is private.

base_class_name

The name of the existing class being inherited from. It must already be defined above this point.

CAREFUL — THE DEFAULT IS PRIVATE

`class Car : Vehicle` and `class Car : private Vehicle` mean exactly the same thing. That is almost never what you want, because it hides everything from the outside. **Write `public` explicitly** unless you have a reason not to. (For a struct the default is the opposite — `public`.)

A complete single-inheritance program

● FULL PROGRAM

```
#include <iostream>
using namespace std;

class Vehicle                // base class
{
public:
    int wheels;
    void applyBrakes()
    {
        cout << "Brakes applied" << endl;
    }
};

class Car : public Vehicle    // derived class
{
public:
    void horn()
    {
        cout << "Beep beep" << endl;
    }
};

int main()
{
    Car c;

    c.wheels = 4;              // inherited data member
    c.applyBrakes();           // inherited member function
    c.horn();                  // Car's own function

    cout << "This car has " << c.wheels << " wheels" << endl;
    return 0;
}
```

OUTPUT

```
Brakes applied  
Beep beep  
This car has 4 wheels
```

IN PLAIN WORDS

Look at `main()`. The object is a `Car`, yet `wheels` and `applyBrakes()` were never written inside `Car` — they arrived through the colon. That is inheritance doing its job: three lines of `Car` gave us a class with four members.

CHAPTER 53

The protected keyword

There is a problem with inheritance and `private`, and `protected` is the answer to it.

The problem

A **private member of a base class cannot be inherited**, so it is not available to the derived class at all. You could of course make it `public` so the derived class can reach it — but then it is reachable by *every* function in the program, and you have thrown away data hiding, which was the point of the class in the first place.

C++ therefore gives a third visibility modifier that serves exactly this limited purpose.

IN PLAIN WORDS — WHAT PROTECTED MEANS

A member declared `protected` is accessible by the member functions **within its own class** and by **any class immediately derived from it**. It cannot be touched by anything outside those two. Think of it as "private to the family".

MODIFIER	OWN CLASS	DERIVED CLASS	OUTSIDE WORLD
<code>private</code>	yes	no	no
<code>protected</code>	yes	yes	no
<code>public</code>	yes	yes	yes

● ALL THREE, SIDE BY SIDE

```
#include <iostream>
using namespace std;

class Base
{
    private:
        int a;
    protected:
        int b;
    public:
        int c;
        Base() { a = 1; b = 2; c = 3; }
        int getA() { return a; }    // the only way out for 'a'
};

class Derived : public Base
{
    public:
        void show()
        {
            // cout << a;    // ERROR - private is not inherited
            cout << "b = " << b << endl;    // fine, protected
            cout << "c = " << c << endl;    // fine, public
            cout << "a = " << getA() << endl;    // fine, through a public method
        }
};

int main()
{
    Derived d;
    d.show();

    cout << d.c << endl;    // fine - public
    // cout << d.b;    // ERROR - protected is not public
    return 0;
}
```

OUTPUT

```
b = 2
c = 3
a = 1
```

What happens to a protected member when it is inherited

Two sentences from the slide, and both are asked:

- A protected member inherited in **public** mode **stays protected** in the derived class. It is therefore usable by that class's member functions *and is still ready for further inheritance*.
- A protected member inherited in **private** mode **becomes private** in the derived class. It is still usable by that class's member functions, but it is **not available for further inheritance**.

WORTH REMEMBERING

The keywords `private`, `protected` and `public` may appear **in any order and any number of times** inside a class declaration. There is no rule that `private` must come first — that is only a convention.

CHAPTER 54

The three modes of inheritance

The visibility mode in `class D : mode B` does not change what the derived class can *use*. It changes what the derived class then *passes on* — to the outside world, and to anything derived from it in turn.

The one table to learn

MEMBER IN BASE CLASS	PUBLIC INHERITANCE	PROTECTED INHERITANCE	PRIVATE INHERITANCE
private	not inherited — inaccessible in every mode		
protected	protected	protected	private
public	public	protected	private

IN PLAIN WORDS — HOW TO REMEMBER IT

The mode acts like a **ceiling**. Nothing can come through more open than the mode you wrote.

With `public` inheritance the ceiling is high, so everything keeps its own level. With `protected` the ceiling is "protected", so anything that was `public` gets pushed down to `protected`. With `private` the ceiling is "private", so everything ends up `private`.

And the top row never changes: **private members of a base class are never inherited**, whatever mode you use.

The two you will actually be asked to compare

PRIVATELY INHERITED

- Public members of the base become **private** members of the derived class
- They can be used by the **member functions** of the derived class
- They are **inaccessible to objects** of the derived class

PUBLICLY INHERITED

- Public members of the base become **public** members of the derived class
- They can be used by the member functions of the derived class
- They **are accessible to objects** of the derived class

● SEEING THE DIFFERENCE

```

class Base
{
    public:
        void hello() { cout << "hello" << endl; }
};

class Pub : public Base { };
class Pri : private Base
{
    public:
        void callIt() { hello(); }    // fine - member function can r
};

int main()
{
    Pub p;
    p.hello();           // FINE - hello() is public inside Pub

    Pri q;
    // q.hello();        // ERROR - hello() is private inside Pri
    q.callIt();          // fine - goes through Pri's own public func
    return 0;
}

```

OUTPUT

```

hello
hello

```

WORTH REMEMBERING

A one-line answer that earns full marks: **the mode decides the access the inherited members get inside the derived class, and therefore how far they can travel onwards; it never gives the derived class access to the base class's private members.**

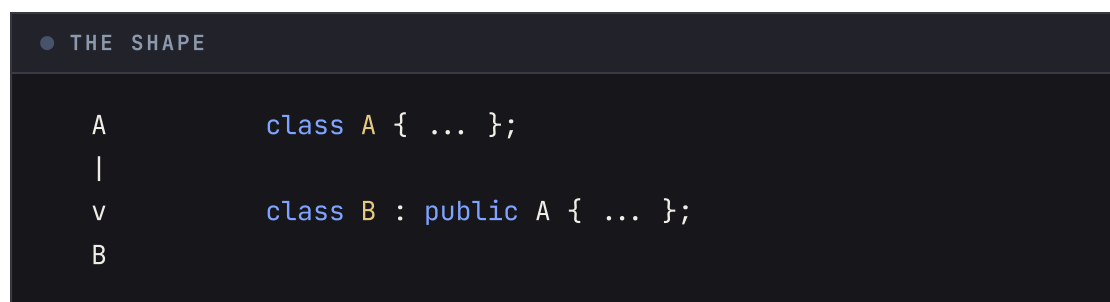
CHAPTER 55

Single and multilevel inheritance

Your slides list five kinds of inheritance. These are the first two, and they are the easiest to draw.

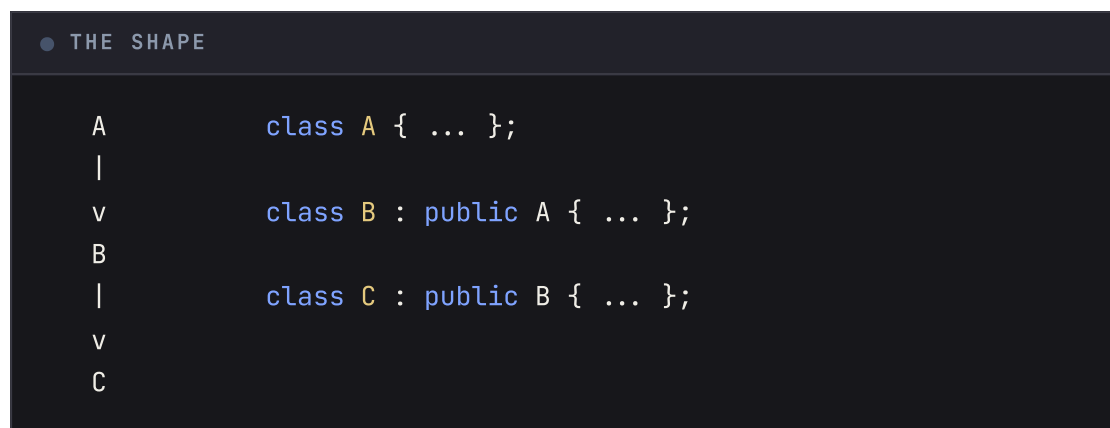
Single inheritance

A class is allowed to inherit from **only one** class — one derived class, one base class. This is the form in Chapter 52.



Multilevel inheritance

The mechanism of deriving a class from another *derived* class. B inherits from A, and then C inherits from B — so C ends up with everything from both.



● THE CLASSIC STUDENT EXAMPLE

```
#include <iostream>
using namespace std;

class Student // level 1
{
protected:
    int roll;
public:
    void setRoll(int r) { roll = r; }
};

class Test : public Student // level 2
{
protected:
    float m1, m2;
public:
    void setMarks(float a, float b) { m1 = a; m2 = b; }
};

class Result : public Test // level 3
{
private:
    float total;
public:
    void show()
    {
        total = m1 + m2;
        cout << "Roll : " << roll << endl;
        cout << "Total : " << total << endl;
    }
};

int main()
{
    Result r;
    r.setRoll(12); // from Student, two levels up
    r.setMarks(78, 65); // from Test, one level up
    r.show(); // its own
    return 0;
}
```

OUTPUT

```
Roll : 12
Total : 143
```

IN PLAIN WORDS — WHY ROLL AND M1 ARE PROTECTED

`Result` uses `roll` and `m1` directly inside `show()`. If they had been `private` in their own classes they would not have been inherited at all and this would not compile. `protected` is what lets data travel down a chain like this, which is exactly the job Chapter 53 described.

Hierarchical inheritance

The third shape: **more than one derived class from a single base class**. The traits of one class are inherited by several others. This is the Vehicle → Bus, Car, Truck picture from Chapter 51.

● THE SHAPE



```

class A { ... };

class B : public A { ... };
class C : public A { ... };
  
```

● SHAPES

```

class Shape
{
    protected:
        float w, h;
    public:
        void setValues(float a, float b) { w = a; h = b; }
};

class Rectangle : public Shape
{
    public:
        float area() { return w * h; }
};

class Triangle : public Shape
{
    public:
        float area() { return 0.5 * w * h; }
};
  
```

Both derived classes reuse `setValues()` unchanged, and each writes only the one function that genuinely differs.

CHAPTER 56

Multiple inheritance

When a class inherits the attributes of **two or more** classes. It lets you combine the features of several existing classes as the starting point for a new one.

● GENERAL FORM

```
class D : mode B1, mode B2, mode B3
{
    // members of D
};
```

● REAL EXAMPLE

```
class Result : public Maths,
               public English
{
    public:
        void total();
};
```

CAREFUL — TWO RULES OF THE SYNTAX

The base classes are separated by a **comma**, and the **access mode must be written for every one of them**. Writing `class D : public B1, B2` silently makes B2 private, because the missing mode defaults to private.

Real examples of the idea

- A child inheriting the physical features of one parent and the intelligence of the other.
- Calculating a result: a `Result` class inherits from `Mathematics` and `English`, and can then reach the data members of both to work out a total.

The order constructors are called in

Straight from the slide: **the constructors of inherited classes are called in the same order in which they are inherited.**

● PROVING IT

```
#include <iostream>
using namespace std;

class Maths
{
public:
    Maths() { cout << "Maths constructor" << endl; }
};

class English
{
public:
    English() { cout << "English constructor" << endl; }
};

class Result : public Maths, public English
{
public:
    Result() { cout << "Result constructor" << endl; }
};

int main()
{
    Result r;
    return 0;
}
```

OUTPUT

```
Maths constructor
English constructor
Result constructor
```

WORTH REMEMBERING

Bases first, in the order they are *listed* after the colon; the derived class's own constructor last. **Destructors run in exactly the reverse order** — Result, then English, then Maths. If you swapped the list to : public English, public Maths the output order would swap with it.

CHAPTER 57

Ambiguity, and the scope resolution fix

The most obvious problem with multiple inheritance, and a guaranteed exam question.

The problem in multiple inheritance

Suppose two base classes have a function with **the same name**, and the derived class does not override it. Call that function through an object of the derived class and **the compiler shows an error** — because it does not know which of the two you meant.

● THIS DOES NOT COMPILE

```
class B1
{
    public:
        void display() { cout << "From B1" << endl; }
};

class B2
{
    public:
        void display() { cout << "From B2" << endl; }
};

class D : public B1, public B2 { };

int main()
{
    D d;
    d.display();      // ERROR - request for member 'display' is amb.
    return 0;
}
```

The fix: say which class you mean

The problem is solved with the **scope resolution operator** `::`, naming the class the function should come from.

● THE FIX

```
int main()
{
    D d;
    d.B1::display();    // the one in B1
    d.B2::display();    // the one in B2
    return 0;
}
```

OUTPUT

From B1
From B2

The other way, which the slide also shows, is to **define a named instance inside the derived class** so that callers never have to think about it:

● RESOLVING IT ONCE, INSIDE D

```
class D : public B1, public B2
{
public:
    void display()    // D's own version
    {
        B1::display();    // decide here, once
    }
};
```

Ambiguity in single inheritance

The same word, a different situation — and your slide is careful to separate them. Here there is no error at all.

● THE DERIVED VERSION WINS

```
class A
{
    public:
        void display() { cout << "From A" << endl; }
};

class B : public A
{
    public:
        void display() { cout << "From B" << endl; }    // same name
};

int main()
{
    B b;
    b.display();           // calls B's version
    b.A::display();        // reaches A's version on purpose
    return 0;
}
```

OUTPUT

From B
From A

IN PLAIN WORDS — THE DIFFERENCE BETWEEN THE TWO CASES

In **multiple** inheritance the two candidates are *side by side*, so neither is preferred and the compiler stops. In **single** inheritance one candidate is *below* the other, and the nearer one simply hides the one above it — so the function in the derived class overrides the inherited one, and a plain call goes to the derived version. You can still reach the base version with `A::`.

WORTH REMEMBERING

Two lines, two situations: **same name in two base classes** → **error**, fix with **`obj.Base::func()`**; **same name in base and derived** → **no error**, the **derived one wins**, use **`obj.Base::func()`** if you want the other one.

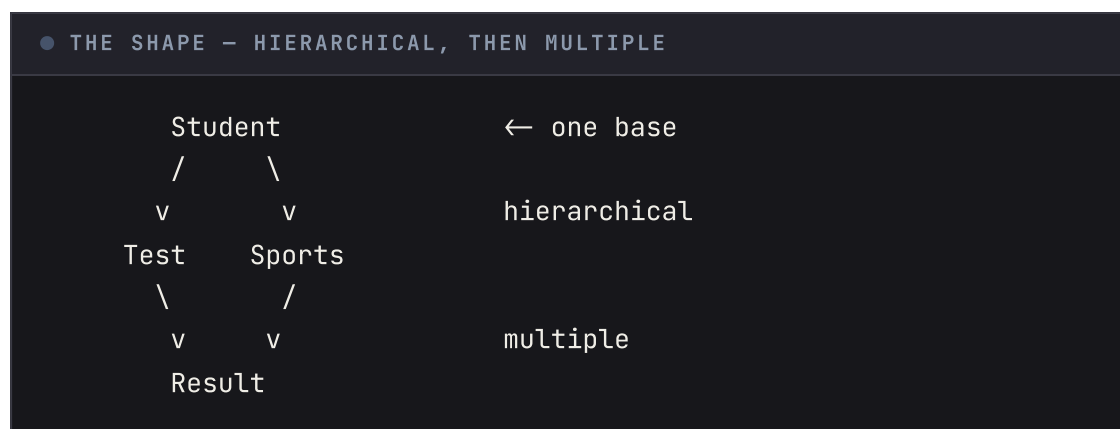
CHAPTER 58

Hybrid inheritance and the virtual base class

The fifth kind, and the one whose name carries a warning inside it.

What hybrid inheritance is

Hybrid inheritance is implemented by **combining more than one type of inheritance** — for example hierarchical inheritance together with multiple inheritance. Your slide titles it *Hybrid (Virtual) Inheritance*, and this chapter explains why that second word is there.



The trouble this shape causes

Test inherits Student. Sports also inherits Student. Result then inherits both — so Result ends up with **two separate copies** of everything Student owns, one arriving through each path.

Now roll is ambiguous again, but this time B1 : : style resolution is hiding a real bug: the object genuinely holds two different roll values, and setting one does not set the other.

The fix: a virtual base class

Mark the shared base `virtual` in *each* of the classes that inherits it directly. C++ then keeps **only one copy** of it, shared by everything below.

A dark-themed presentation slide. At the top, a dark grey header bar contains the text '● FULL PROGRAM' in white. The main body of the slide is a large, solid dark blue rectangle, which is slightly lighter than the background, creating a subtle contrast. The overall aesthetic is minimalist and professional.

● FULL PROGRAM

```
#include <iostream>
using namespace std;

class Student
{
    protected:
        int roll;
    public:
        void setRoll(int r) { roll = r; }
};

class Test : virtual public Student    // note: virtual
{
    protected:
        float marks;
    public:
        void setMarks(float m) { marks = m; }
};

class Sports : virtual public Student    // note: virtual
{
    protected:
        float score;
    public:
        void setScore(float s) { score = s; }
};

class Result : public Test, public Sports
{
    public:
        void show()
        {
            cout << "Roll : " << roll << endl;    // only ONE ro
            cout << "Total : " << marks + score << endl;
        }
};

int main()
{
    Result r;
    r.setRoll(21);
    r.setMarks(78);
    r.setScore(9);
    r.show();
}
```

```
    return 0;
}
```

OUTPUT

Roll : 21
Total : 87

CAREFUL

Remove the two `virtual` keywords and this program **stops compiling** — `roll` inside `show()` becomes ambiguous. The word `virtual` has to be written on the *intermediate* classes (Test and Sports), not on Result, because they are the ones doing the duplicating.

WORTH REMEMBERING

The shape is often called the **diamond problem**. The one-line answer: **when two classes derived from the same base are themselves inherited by one class, that class gets two copies of the base; declaring the base `virtual` keeps a single shared copy and removes the ambiguity.**

The five kinds, on one page

KIND	SHAPE	IN ONE LINE
Single	$A \rightarrow B$	One derived class, one base class.
Multilevel	$A \rightarrow B \rightarrow C$	A class derived from another derived class.
Hierarchical	$A \rightarrow B, A \rightarrow C$	Several derived classes from one base class.
Multiple	$A, B \rightarrow C$	One derived class from several base classes.
Hybrid	a mix	Two or more of the above combined; usually needs a virtual base.

CHAPTER 59

Inheritance: the whole thing on one page

Nine things. If you can say all nine without looking, you are ready for any inheritance question on the paper.

1 What it is.

Deriving ~~geus~~ability

a

new

class

from

an

existing

one,

so

the

new

class

gets

the

old

one's

properties

without

changing

it.

Also

called

derivation.

The

point

is

.

2 The names.

Old is-a

=

base

/

parent

/

super.

New

=

derived

/

child

/

sub.

The

test

for

using

it at

all is

.

3 The syntax.

class — the colon means "derived from", and the mode defaults to

D :

mode

B {

};

private if you leave it out.

4 Private members are never inherited

, in protected
any
mode.

exists
so
that
data
can
be
hidden
from
the
world
but
still
reach
the
derived
class.

5 The mode is a ceiling.

public
keeps
levels
as
they
are;
protected
pushes
public
down
to
protected;
private
pushes
everything
to
private.

6 Five kinds:

single,
multilevel,
hierarchical,
multiple,
hybrid.

7 Constructor order

in
multiple
inheritance:
base
classes
first,
in
the
order
listed
after
the
colon;
destructors
in
reverse.

8 Ambiguity.

Same `obj.Base::func()`
name
in
two
bases
→
compiler
error,
fixed
with

.
Same
name
in
base
and
derived
→
no
error,
derived
wins.

9 The diamond.

Two virtual
copies
of a
shared
base;
declare
it

on
the
intermediate
classes
to
keep
one.

Quick self-test

? A base class has a private member `x`. Can a publicly derived class use it?

No. Private members of a base class are never inherited, whatever the mode. The derived class can only reach `x` indirectly, through a public or protected member function of the base that returns it. If you want the derived class to touch it directly, declare it protected instead.

? What is the access of a public base member after `class D : protected B`?

protected. The mode is a ceiling: nothing comes through more open than the mode. So a public member is pushed down to protected, and a protected member stays protected. Objects of `D` can therefore no longer call it, but anything derived from `D` still can.

? What is printed, and in what order?

```
class A {
public: A()
{cout<<"A";}
~A()
{cout<<"a";}
};

class B {
public: B()
{cout<<"B";}
~B()
{cout<<"b";}
};

class C :
public B,
public A {
public: C()
{cout<<"C";}
~C()
};

int
with main()
{ C
obj; }
```

```
{cout<<"c";}
};
```

BAC cab

Construction follows the order in the base list, which is B then A — not alphabetical order, and not the order the classes were declared in. So B, A, then C's own constructor: **BAC**.

Destruction is always the exact reverse: C, then A, then B: **cab**.

? **Why does** `class Car : Vehicle` **often fail to compile in** `main()` **even though it compiles as a class?**

Because the missing mode defaults to **private**. The class itself is perfectly legal, and its own member functions can use everything they inherited — but every public member of `Vehicle` has become private inside `Car`, so `main()` can no longer call any of them through a `Car` object. Writing `public` explicitly fixes it.

? **Two classes** `Test` and `Sports` **both inherit** `Student`, **and** `Result` **inherits** `Student` **both.** **How many copies of**

Two — one through each path — which is why any member of `Student` is ambiguous inside `Result`.

Declaring `Student` a **virtual base class** in both `Test` and `Sports` (`: virtual public Student`) reduces that to **one shared copy** and removes the ambiguity. This is the hybrid, or diamond, case.

PART X

Revision

Everything compressed. Read this the night before; read the chapters the week before.

CHAPTER 60

One-page cheat sheet

Every piece of syntax in this course

● CLASS AND OBJECTS

```
class ClassName
{
    int data;                // private by default
public:
    void func(int a);        // declaration
    void inlineFunc() { /* body */ } // defined inside = auto inline
};

void ClassName :: func(int a) // defined outside
{
    data = a;
}

ClassName obj;                // create an object
ClassName arr[10];            // array of objects
obj.func(5);                  // call a member function
```

● CONSTRUCTORS AND DESTRUCTOR

```
class X
{
    int m;
public:
    X()          { m = 0; }          // default constructor
    X(int a)     { m = a; }          // parameterized
    X(X &other)   { m = other.m; }    // copy constructor (must be .
    ~X()         { }                // destructor – no args, no r

};

X a;            // default
X b(10);        // parameterized, implicit
X c = X(10);    // parameterized, explicit
X d(b);         // copy constructor
X e = b;        // copy constructor (copy initialization)
d = b;          // assignment, NOT the copy constructor
```

● STATIC MEMBERS

```
class X
{
    static int count;                // declared inside
public:
    static void show();              // static member function
};

int X :: count;                     // DEFINED outside – compulso

X :: show();                        // call without an object
```

● FRIENDS

```
class X
{
    friend void f(X);                // a free function
    friend int Y :: g();             // one member function of cla
    friend class Z;                  // every member function of Z
};

void f(X obj) { obj.privateMember; } // no 'friend', no 'X::'
```

● CONST MEMBER FUNCTION

```
int getvalue() const;           // declaration

int X :: getvalue() const      // definition – const in BOTH
{
    return m;                 // may not modify anything
}
```

● POINTERS TO MEMBERS

```
int X :: *ptr = &X :: m;      // pointer to a data member
obj.*ptr    = 10;             // via an object
objPtr->*ptr = 20;             // via a pointer to object

int (X::*fp)(float) = &X::f;  // pointer to a member functi
(obj.*fp)(3.5);              // brackets are COMPULSORY
(objPtr->*fp)(3.5);
```

● OPERATOR OVERLOADING

```
// member function
return-type ClassName :: operator op (arglist) { }

X X::operator + (X b)    { }    // binary, member : 1 argumen
void X::operator - ()    { }    // unary, member : 0 argumen
void X::operator ++ ()   { }    // prefix ++x
void X::operator ++ (int){ }    // postfix x++ (dummy int)

// friend function
friend X operator + (X a, X b);  // binary, friend : 2 argumen
friend X operator - (X a);      // unary, friend : 1 argumen
```

● FUNCTIONS

```
type name(arglist);           // prototype
inline type name(arglist) { }  // inline function
int sum(int a, int b, int c = 0); // default args – in PROTOTYP

void f(int x);                 // call by value
void f(int *x);                // call by address → f(&a)
void f(int &x);                 // call by reference → f(a);
int &max(int &a, int &b);       // returns a reference
```

● MEMORY AND I/O

```

int *p = new int;           // allocate one
int *p = new int(25);       // allocate and initialise
int *p = new int[10];       // allocate an array
delete p;                   // free one
delete [] p;                // free an array

cout << x << endl;         // output
cin  >> x;                  // input
cout << setw(10) << setfill('*') << x; // needs <iomanip>
cout << fixed << setprecision(2) << x; // needs <iomanip>

int &ref = variable;        // reference variable
::globalVar;                // scope resolution

```

The lists you must memorise

LIST	THE ANSWER
Number of C++ keywords	63
Six core OOP concepts	Classes, Objects, Inheritance, Abstraction, Encapsulation, Polymorphism
Four paradigms	Imperative, Functional, Object, Logic
Five kinds of token	Keywords, Identifiers, Constants, Strings, Operators
Five storage classes	auto, extern, static, register, mutable
Three control structures	Sequential, Selection, Loop
Cannot be overloaded at all	:: . .* ?:
Must be a member , never a friend	= () [] →
Three pointer-to-member operators	::* .* →*
Three ways to pass an argument	By value, by address, by reference
Four kinds of constructor	Default, Parameterized, Copy, Dynamic

CHAPTER 61

Mistakes people make

Twelve errors that cost marks and hours. Each one is drawn from a rule earlier in this book.

#	THE MISTAKE	THE FIX
1	Missing the semicolon after a class: <code>}</code>	A class declaration ends <code>};</code> — see Ch. 36
2	Trying to read a private member from outside: <code>x.number = 100;</code>	Go through a public member function — Ch. 36
3	Declaring a static member but never defining it outside the class	Add <code>int X::count;</code> at file scope — Ch. 40
4	Writing a copy constructor as <code>A(A)</code>	It must take a reference: <code>A(A&)</code> — Ch. 46
5	Giving a constructor a return type	Constructors and destructors have no return type, not even <code>void</code> — Ch. 45
6	Writing only a parameterized constructor, then using <code>X a;</code>	Once you write any constructor, the free default one disappears — Ch. 45
7	Calling <code>new</code> in the constructor and never calling <code>delete</code>	Every <code>new</code> in a constructor needs a matching <code>delete</code> in the destructor — Ch. 47
8	Using <code>delete p;</code> on memory made with <code>new int[10]</code>	Use <code>delete [] p;</code> — Ch. 24
9	Trying to overload two functions that differ only in return type	They must differ in type, number or order of <i>parameters</i> — Ch. 35
10	Putting default argument values in the function definition	They belong in the prototype — Ch. 33
11	Writing <code>A = 2 + B;</code> with a member operator function	Use a friend function — <code>2</code> has no member functions — Ch. 49
12	Forgetting <code>#include <iomanip></code> before using <code>setw</code>	Only <code>endl</code> comes free with <code><iostream></code> — Ch. 25

CAREFUL — THREE PAIRS THAT GET CONFUSED

- **Structure vs Class:** they are almost identical in C++. The one real difference is that members of a `struct` are **public by default**, while members of a `class` are **private by default**.
- **Encapsulation vs Abstraction:** encapsulation is the *mechanism* of hiding (using `private`); abstraction is the *purpose* — showing only what the user needs.
- **Copy constructor vs Assignment:** `X b = a;` creates `b`, so the copy constructor runs. `b = a;` when `b` already exists is assignment.

CHAPTER 62

Test yourself

Twenty questions drawn from every part of this book. Try to answer each one before opening it.

PART I & II — FOUNDATIONS

? Who created C++, and what was it originally called?

Bjarne Stroustrup created C++. It began life as "**C with Classes**" — the aim was to keep everything useful about C and add the ability to bundle data together with the functions that operate on it.

? Name the four programming paradigms and say what a program "is" in each.

- **Imperative** — a program is Algorithms + Data
- **Functional** — a program is Functions
- **Object** — a program is Data
- **Logic** — a program is Facts + Rules + Queries

? What is the single biggest weakness of procedure oriented programming?

Global data is exposed. Most functions share global data, and data moves openly around the system from function to function. Any function can change any piece of data, so a bug in one function can break an unrelated part of the program. Object oriented programming exists to fix exactly this.

? List any five features of object oriented programming.

- Emphasis is on data rather than procedure.
- Programs are divided into objects.
- Data structures are designed such that they characterise the objects.

- Functions that operate on the data of an object are tied together in the same data structure.
- Data is hidden and cannot be accessed by external functions.
- Objects may communicate with each other through functions.
- New data and functions can be easily added.

PART III & IV — TOKENS, TYPES, OPERATORS

? What is the difference between a structure and a union in terms of memory?

The size of a **structure** is greater than or equal to the **sum** of the sizes of its members — each member gets its own unique storage location. The size of a **union** is the size of its **largest** member, because all the members share the same piece of memory.

The consequence: in a structure, altering one member does not affect the others. In a union, altering any member alters the other members' values, and only one member can be accessed at a time.

? What will `enum color {red = 5, blue, green};` assign to each name?

`red = 5`, `blue = 6`, `green = 7`. Once you assign a value to one enumerator, the following ones continue counting up from there.

? Why is `*ip = *gp;` invalid when `gp` is a `void*` ?

A generic pointer can be *assigned* a pointer value of any basic data type, **but it may not be dereferenced**. It would make no sense to dereference a pointer to a void value — the compiler has no idea how many bytes to read or how to interpret them.

So `gp = ip;` is valid, but `*ip = *gp;` is not.

? Give the initial value of a variable in each of the five storage classes.

STORAGE CLASS	LIFETIME	VISIBILITY	INITIAL VALUE
<code>auto</code>	Function block	Local	Garbage
<code>extern</code>	Whole program	Global	Zero
<code>static</code>	Whole program	Local	Zero
<code>register</code>	Function block	Local	Garbage
<code>mutable</code>	Class	Local	Garbage

The pattern: anything whose lifetime is the whole program starts at zero; anything short-lived starts as garbage.

? What does `::m` mean, and what problem does it solve?

`::` is the **scope resolution operator**. `::m` means the **global** version of `m`, not the local one declared in the current block.

It solves a problem C had: in C, the global version of a variable cannot be accessed at all from within an inner block where a local variable of the same name hides it. C++ added `::` to uncover that hidden variable.

? Give three differences between `malloc/free` and `new/delete`.

- `malloc` is a **library function**; `new` is an **operator** built into the language.
- `malloc` needs you to calculate the size and usually cast the result; `new` works out the size and returns the correct pointer type.
- `new` **calls the constructor** and `delete` **calls the destructor**; `malloc` and `free` do neither.

PART V — FUNCTIONS

? Write a program to swap two numbers by call by reference.

● ANSWER

```
#include <iostream>
using namespace std;

void swap(int &a, int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int x = 10, y = 20;
    swap(x, y);
    cout << "x = " << x << ", y = " << y;
    return 0;
}
```

The formal arguments `a` and `b` become **aliases** for `x` and `y`. Note that the call looks identical to call by value — only the function header reveals the difference.

? Why can an inline function be ignored by the compiler, and when should you use one?

`inline` is a **request, not a command**. The compiler may decline it — for example when the function contains a loop, a switch, or is recursive.

Use it for **small, frequently called functions** where the call overhead is significant compared to the function's execution time — simple getters and setters, or small mathematical operations. The benefit diminishes as the function grows, and a large inlined function called many times causes **code bloat**.

? Why is `int sum(int a, int b = 5, int c);` illegal?

Default values can be assigned only for trailing arguments — never to leading arguments or arguments in the middle.

The reason: arguments are matched left to right. If someone wrote `sum(1, 2)`, the compiler could not tell whether 2 was meant for b or for c. The arguments you may omit must all be at the right-hand end.

? **Can two functions be overloaded if they differ only in return type? Explain.**

No. Overloaded functions must differ in the **type, number or order of parameters**. A difference only in return type is not enough.

The reason: consider a call written on its own line with the result discarded. The compiler sees only the arguments, which are identical for both versions, so it cannot choose.

Related trap: `typedef int INT;` does **not** create a new type, so `display(int)` and `display(INT)` are also not overloadable. But `display(char*)` and `display(const char*)` **are**, because those are genuinely different types.

PART VI & VII — CLASSES, FRIENDS, POINTERS

? **If I create 10 objects of a class, how many copies of the member functions exist?**

One. Member functions are created and placed in memory only once, when the class is defined, and all objects share that single copy.

Member *variables* are different: space for them is allocated **separately for each object**, because they hold different values in different objects.

? **Why must a static data member be defined outside the class?**

Inside the class you only **declare** that the static member exists — no storage is allocated. The definition outside, written as `int item :: count;`, is what actually allocates the storage.

Leave it out and the program compiles but fails at **link time** with "undefined reference to `item :: count`".

Remember also: a static member is initialised to zero when the first object is created, there is only one copy shared by all objects, and it is visible only within the class but lives for the entire program.

? A friend function breaks encapsulation. Why does C++ allow it?

Because some operations are genuinely **symmetric** between two classes, or between a class and a built-in type, and forcing them to be a member of one side would be artificial.

The crucial safeguard: **friendship is granted by the class itself, in its own declaration**. A function cannot declare itself a friend. So the class still controls exactly who gets in. It is a controlled exception, not a hole.

Three properties to quote: friendship is **granted, not taken**; it is **not mutual**; and it is **not inherited**.

? What are the three pointer-to-member operators and what does each do?

OPERATOR	PURPOSE
<code>::*</code>	To declare a pointer to a member of a class
<code>.*</code>	To access a member using an object name and a pointer to that member
<code>→*</code>	To access a member using a pointer to the object and a pointer to that member

● ALL THREE IN USE

```
int Data::*ptr = &Data::a;    // ::*
d.*ptr    = 10;               // .*
dp→*ptr = 20;                 // →*
```

PART VIII — CONSTRUCTORS AND OPERATORS

? Why must a copy constructor take its argument by reference?

Passing an object **by value** means making a copy of it, and making a copy means calling the copy constructor. So `A(A)` would have to call itself in order to build its own argument — which would call itself again, forever.

Therefore `A(A);` is **illegal** and `A(A&);` is **legal**.

? In what order are objects destroyed, and when exactly?

In the reverse order of their creation, and at the closing brace of the block they were declared in.

So if `A1`, `A2`, `A3` are created in `main` and `A4` is created inside an inner block, then `A4` is destroyed first — at the end of the inner block — and then `A3`, `A2`, `A1` are destroyed in that order when `main` ends.

? Which four operators cannot be overloaded, and which four must be members?

Cannot be overloaded at all:

`::` `.` `.*` `?:`

Can be overloaded, but only as a member function — never as a friend:

`=` `()` `[]` `→`

These are two different lists and both are commonly asked. Do not mix them up.

? Why does `A = 2 + B;` fail with a member operator function but work with a friend?

For a **member** operator function, `a + b` is translated into `a.operator+(b)`. The left operand becomes the calling object.

So `2 + B` would become `2.operator+(B)`. But `2` is an `int`, and an `int` has no member functions. Compile error.

For a **friend** function, `a + b` becomes `operator+(a, b)` — an ordinary two-argument call, with no requirement that the first argument be an object.

Both orders work.

Note also the argument counts: a friend function always needs **one more argument** than the member version. Unary: 0 as member, 1 as friend. Binary: 1 as member, 2 as friend.

? Write a class that safely manages a dynamically allocated string.

● ANSWER – THE THREE ESSENTIALS

```
class String
{
    char *p;
    int len;

public:
    String(const char *s)           // 1. constructor: TAKE memory
    {
        len = strlen(s);
        p = new char[len + 1];
        strcpy(p, s);
    }

    String(const String &s)         // 2. copy constructor: deep copy
    {
        len = s.len;
        p = new char[len + 1];
        strcpy(p, s.p);
    }

    ~String()                      // 3. destructor: GIVE IT BACK
    {
        delete [] p;
    }
};
```

Any class that allocates memory with `new` needs its own copy constructor and destructor. Without the copy constructor, copying a `String` would copy the *pointer*, so two objects would point at the same memory — and when both destructors ran, that memory would be deleted twice and the program would crash.

Programs to write yourself

FROM YOUR COURSE MATERIAL

- Write a C++ program to swap the values of two numbers using call by value, call by address, and call by reference variable.
- Write a program to compute the area of a square, a rectangle and a circle by overloading the `area()` function.
- Write a function `power()` to raise a number m to a power n . Take a `double` for m and an `int` for n . Use a default value of 2 for n so the function calculates the square when that argument is omitted. Write a `main` that reads m and n from the user. Use function overloading.

THREE MORE, TO TIE THE PARTS TOGETHER

- Write a class `Distance` with feet and inches. Overload `+` as a member function to add two distances, and overload `-` as a unary operator to negate one.
- Write a class that uses a static data member to count how many objects currently exist. Increment in the constructor, decrement in the destructor, and print the count using a static member function.
- Write a class `Matrix` that allocates a 2D array in its constructor using `new`, frees it in its destructor, and overloads `+` as a friend function to add two matrices.

The C++ Objects Handbook

Built from the eight course decks for Object Oriented Programming (CS363), DoCSE, SVNIT Surat – Dr. Balu L. Parne, after E. Balagurusamy and Yashwant Kanetkar.

Covers chapters 1-62 · every code example is taken from or built directly on the material.